Self-Reconciling Data Pipeline — Architecture Design v5

22 Oct 2026 · Bhavna Rathi

What changed in v5 and where the build stands

v5 combines the full v4 architecture with the build plan. Part A reproduces v4 unchanged; v5 adds a design plan for each of the seven POC phases, the actual build status from the runbook, and a frozen Phase 5 (agents) design. Policy stays at 1.7.0 until Phase 4 adds rules.

How this document is organised

- **Front matter (this section):** what changed, the four status states and the gate status.
- **Part A — Architecture v4 (unchanged):** the complete v4 design, all 17 sections, with its own page numbers. Items marked *v4* there were new in v4. Where Part A conflicts with Parts B and C (agent scope, value-level evidence mechanism, verification tracker, trial-account platform), **Parts B and C apply**.
- **Part B — Phase plans:** components mapped to phases, POC differences, and one plan per phase.
- **Part C — Tracking:** the v5 verification tracker, SLOs, open decisions and change log. The v5 tracker supersedes the v4 tracker in Part A §13. The Phase 5 exit gate is a test-to-evidence matrix: a group passes only with recorded output.

What v5 adds to v4

- Part B: one plan per phase, with the same eight headings: status and exit gate · goal and why, with an example · design · GCP infrastructure · done so far or build checklist · test cases · issues and fixes · risks and differences from plan.
- Components mapped to phases (Part B) and to the ten steps of the GCP POV deck.
- Phase 5 frozen: three advisory agents in scope (Runbook match-only), an approval state machine with a proposal fingerprint, a minimal agent input contract, and a ten-group exit gate.
- Verification tracker updated item by item from recorded test evidence only.
- One recorded design deviation from Phase 2 (value-level evidence written through a model, not `store_failures`).
- Two build findings: the KMS role must be `cryptoKeyEncrypterDecrypterViaDelegation`, and dbt models need `+schema: audit`.

Four separate states. This document never merges them:

State	Means	Proven by
Implemented	Code and infrastructure exist	A commit or Terraform apply
Tested	The test has been run	A recorded run output
Passed	The phase's exit-gate tests all passed	Recorded results per gate item
Approved for healthcare data	Cleared for real patient data	Written sign-off (residency, BAA, privacy review)

No phase is approved for healthcare data. All work so far uses synthetic data only.

Gate status as of 22 Oct 2026 (project `reconcile-poc-29056`, asia-south1)

Phase	Implemented	Exit gate	Evidence
0 Foundations	Yes	3 of 4 passed (20 Oct). Destroy + rebuild proof pending, blocked by 7-day archive retention until about 28 Oct	CI green on `0a08808`; `/api/health` ok
1 Ingest and archive	Yes	Passed 21 Oct (G1–G5, dates 11–13 Oct)	`make verify` output; fix `3e37dac`
2 Models and checks	Yes	Passed 22 Oct	Runbook v1.2 demos T2–T7, T11
3 Publication gate	No	Not started	—
4 Decision engine	No	Not started	—
5 Agents and approvals	No	Not started	—
6 Breaker and demo	No	Not started	—

The Phase 2 exit gate and the v4 verification tracker are different lists. The gate has passed; the tracker is only partly verified (see the tracker section at the end).

Part A

Architecture design v4 — reproduced unchanged

The following pages are the complete v4 architecture (14–15 Oct 2026, policy 1.7.0), sections 1–17. Page numbers on those pages are v4's own. For current POC scope and status, Parts B and C supersede any conflicting statement on these pages; the verification tracker in Part C supersedes the one in v4 §13.

Self-Reconciling Data Pipeline with Recovery Decision Engine and Remediation using dbt

Architecture design · Healthcare data platform · 14 October 2026 · revised 15 October 2026 (v4) · pilot implementation design

1. Problem statement
2. Challenges today
3. Our approach
4. Platform and infrastructure on Google Cloud
5. End-to-end flow
6. Inputs and outputs
7. Failures we must handle
8. Archive and orphaned records
9. Decision engine
10. Remediation playbook
11. Role of dbt
12. Multi-agent roles
13. Guardrails and production-readiness controls
14. End-to-end walkthrough: one night
15. Components: functionality, inputs and outputs
16. Business benefits by stakeholder
17. Sources

> **What changed in v4.** v4 adds five safeguards and their implementation design to v3, with no architectural redesign. Changed or added material is marked v4.
>
> - **Upstream-outage breaker:** correlated failures from one source become one held, grouped incident instead of many separate fixes (sections 9, 10, 13).
> - **Deployment context:** recent releases are attached to incident evidence as a clue, never as proof (sections 6, 9).
> - **Value-level reconciliation:** critical clinical fields are compared record by record using keyed values, with no values or digests stored (sections 7, 11).
> - **Agent provenance:** every agent call is traceable to its agent, model, prompt and policy versions (section 12).
> - **Human-approved learned thresholds:** thresholds proposed from history run in shadow and need owner approval (section 12).
> - **Pre-pilot essentials:** AI residency and coverage confirmation (section 4), an evaluation gate for agent changes and a defined agent-outage path (section 12), duplicate-event handling and policy validation in CI (section 13).
>
> Policy moves to version 1.7.0. The snapshot-aligned control totals, quarantine accounting, raw-archive replay, five gates and action catalog are unchanged. **Status:** this is a pilot implementation design with defined verification criteria, not a production-validated implementation. The SQL and the new controls are a reference implementation; the verification tracker in section 13 marks what has passed testing, and nothing is described as verified until its tests pass.

1. Problem statement

Our clinical and billing pipelines can prove that they ran, but not that the data they delivered is correct. Every night, patient, encounter, lab result and claims data moves from the EHR and revenue-cycle systems into the data warehouse, where dbt builds the models behind admissions, bed-occupancy, revenue-cycle and quality reporting. Each job reports success or failure, yet nothing checks the result against the source. A load can finish green with encounters missing, claims duplicated, adjustments never applied or lab values in the wrong unit, and the first sign is often a wrong figure noticed by a clinician, a billing analyst or the quality team.

The source data also keeps changing underneath the pipeline. Claims are reversed and re-adjudicated after they are loaded, sometimes weeks later and outside any short reprocessing window; late documentation and delayed lab results cross the day boundary, and source systems and facilities define “a day” in different time zones. Under retention policy, older encounters and claims are archived out of the operational systems, and duplicate patients are merged in the master patient index. Both leave **orphaned records**: lab results, medications, payments and adjustments that point at encounters or patient IDs the live source no longer holds. A naive comparison therefore flags correct history as missing and can miss real errors entirely.

When a real problem is found, recovery is ad hoc and manual. L1 support searches Splunk logs, opens the git runbook and runs validation queries by hand within a 30-minute window, then escalates to L2 and L3, which have no agreed response target yet. Nothing decides what kind of failure it is or which fix is safe, fixes are not re-verified against the source, and some make things worse: a full refresh can erase history the source has already archived. Meanwhile unverified data may already be in clinical, financial or regulatory use.

We also cannot measure the problem today: there is no reliable count of data incidents per month, of how long they take to detect and fix, or of how many are first found by users. The pilot will establish this baseline from Splunk.

Goal: a self-reconciling, self-healing data pipeline. It proves every load against its source before publishing it, tells real failures apart from expected differences such as archived history, patient merges and late records, fixes well-understood problems on its own and verifies each fix, holds back anything it cannot verify, and records every decision for audit. Everything below is a proposed design, to be proven in a pilot.

Scope boundary: reconciliation proves that the warehouse matches its source. It does not prove that the source itself is correct; anomaly and value-range checks give partial cover for that.

2. Challenges today

Challenge	What happens	Impact
No source reconciliation	Jobs are checked for completion, never against source counts and totals	A green run is taken as correct data
Silent data failures	Encounters missing, claims duplicated, adjustments not applied, lab units changed	Wrong figures reach clinical, finance and quality reports
Source changes break naive checks	Claim reversals after load, late documentation, UTC vs local-time day boundaries	False alarms on correct data; real errors missed
Archiving and patient merges	Archived encounters and merged patient IDs leave child records pointing at missing parents	Orphaned labs and payments; history looks “extra”
Manual triage	L1 searches Splunk, reads the git runbook and runs queries by hand	Slow, inconsistent fixes that depend on who is on call
Escalation delays	L1 has 30 minutes; L2 and L3 have no agreed response target, so issues can wait in queues	High MTTR; stale data for hours
Unverified, ad hoc recovery	Fixes chosen case by case and not re-checked; full refreshes risk losing archived history	Slow, inconsistent recovery that can make data worse
No verification gate	Data is published before it is checked	Bad data spreads to dashboards and submissions
Repeated failures	The same issues (late feeds, partial loads, double loads) recur	Lost engineering time; no learning
Weak audit trail	Fixes are not recorded consistently	Hard to show what changed in a figure, when and why

3. Our approach: what we bring

We add a self-healing control loop around the existing pipeline: **detect → decide → fix → verify**. It follows the four stages used by self-healing pipeline tools (detection, diagnosis, remediation and validation), with two differences: it checks data correctness against the source, not only job failures, and the audit schema acts as the sandbox where every

fix is applied and re-checked before anything is published. It plugs into the tools and process the team already uses: Airflow, dbt, Splunk, Slack, git runbooks and the L1/L2/L3 support levels.

- **Self-reconciliation:** every load is checked against source control totals per business day (record counts, distinct keys such as encounter ID and claim ID, and amounts), not just job status.
- **Value-level reconciliation for critical fields** v4: where a dataset declares critical clinical fields, each record's values are compared between the raw batch and the warehouse using keyed values, so a corrupted value is caught even when counts and totals match.
- **Expected-difference awareness:** archived history, orphans whose parent is archived or merged, late records and gaps within tolerance are recognised and logged, never "fixed".
- **Write-Audit-Publish:** data builds in an audit schema and is published only after the checks pass.
- **Replay from a raw archive:** every extract is kept immutably, so recovery works even after the source has archived or purged the records.
- **Deterministic decision engine:** rules classify each failure, score its risk and choose one action. The same input always gives the same, explainable decision.
- **Shared-outage awareness** v4: when many datasets from one source fail together, the engine treats it as one upstream outage, holds them as a group and sends one alert, instead of attempting many separate fixes.
- **Deployment-aware diagnosis** v4: recent releases that touched the failing models are attached to the evidence as a clue, never as proof, so code regressions are not "fixed" by replaying data.
- **dbt as eyes and hands:** dbt tests detect problems, and dbt builds carry out every fix that changes warehouse data; the orchestrator handles timing, publication and people. The engine decides which fix runs and when.
- **Bounded multi-agent support:** four specialised agents diagnose, find runbooks, communicate and draft code fixes. They suggest; the engine decides. They have no write access to production and see only masked data. Every agent call is traceable to its agent, model and prompt versions v4.
- **Three response modes, mapped to support levels:** auto-fix for safe, reversible fixes (the engine acts as L1, within the 30-minute L1 budget); approve, where the data owner (L2) confirms a proposed fix in Slack; and engineer fix for code, schema or logic problems, handed to L3 with evidence. Every incident also gets a severity (P1, P2 or P3) that sets its urgency.
- **Verify every fix:** an incident closes only when reconciliation passes again.
- **Wide periodic reconciliation:** a weekly check of the last 90 days, by amount and distinct keys, catches late reversals and corrections outside the daily lookback window and triggers targeted rebuilds of the affected dates.
- **Dependency-aware publishing:** a mart is published only when every dataset it depends on has passed; otherwise it stays on its last verified day, so no report mixes verified and unverified data.
- **Per-dataset hold policy:** each dataset declares whether a known, small gap may be published with a visible caveat, and who approves it. By default, data is held.
- **Guardrails enforced by infrastructure, not only code:** only the publish step can write to analytics, agents can read only masked data, and every query is audit-logged (section 4).
- **Built on Google Cloud with a cloud-agnostic core:** BigQuery, Cloud Storage, Managed Airflow and Cloud Run underneath; Airflow, dbt Core, Python and SQL on top, so the warehouse or cloud can change without redesign.
- **Learning loop with an approval boundary:** every incident is audited, including auto-healed ones. Resolutions and closed tickets lead to proposed runbook, rule or threshold changes v4, which are tested against past incidents and failure-injection cases and approved by a human before they can run automatically.

4. Platform and infrastructure on Google Cloud

dbt does not store or process data itself: it compiles SQL and runs it inside a warehouse. The audit and production datasets, the publish step, stored test failures, quarantine tables, snapshots and the ops state store all live on the platform. Reconciliation and transformation must never run on the EHR or billing production databases, which serve clinical operations.

Decision: the platform is **Google Cloud**, in region `asia-south1` (Mumbai), with **BigQuery** as the warehouse. The core of the design stays cloud-agnostic: Airflow, dbt Core, Python and SQL. SQL examples in this document use BigQuery syntax.

Component	Google Cloud service	Cloud-agnostic alternative

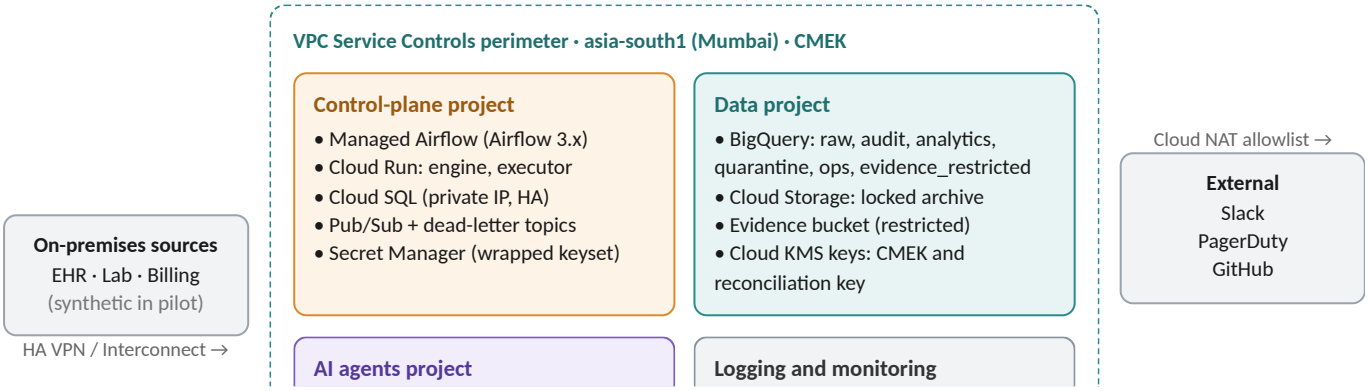
Warehouse	BigQuery, with datasets `raw`, `audit`, `analytics`, `quarantine`, `ops` and `evidence_restricted` v4; tables partitioned by business date and clustered by key	Snowflake, Redshift or Databricks
Transform and test	dbt Core with the `dbt-bigquery` adapter, run from Airflow through Cosmos	Same models; change the adapter and `profiles.yml`
Raw archive	Cloud Storage with Bucket Lock retention, object versioning and lifecycle tiers (Nearline → Coldline → Archive)	S3 Object Lock or ADLS immutable storage
Orchestrator	Managed Service for Apache Airflow (formerly Cloud Composer), Gen 3, with an Airflow 3.x build selected explicitly (the default build is still Airflow 2)	Self-hosted Airflow on any cloud or Kubernetes
Decision engine and executor	Cloud Run services and jobs (Python containers)	Any container runtime
Ops state and locks	Cloud SQL for PostgreSQL, private IP, high availability	Any PostgreSQL
AI agents	Gemini Enterprise Agent Platform (successor to Vertex AI): Agent Development Kit, Agent Runtime, Model Armor	ADK is open source; the model can be swapped
Observability and messaging	Cloud Logging, Cloud Monitoring, Pub/Sub, Knowledge Catalog lineage (formerly Dataplex); Slack, PagerDuty and GitHub; Splunk optional	Splunk, OpenLineage
Security	IAM, Secret Manager, Cloud KMS (including the KMS-wrapped keyset for value-level reconciliation v4), VPC Service Controls, Sensitive Data Protection, BigQuery policy tags, dynamic masking and row-level security	Equivalent controls on any cloud

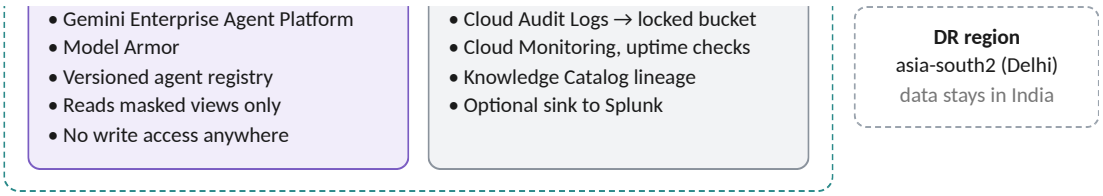
Write-Audit-Publish on BigQuery. BigQuery has no atomic schema swap, so the publish step clones each verified table from `audit` into `analytics` (`CREATE OR REPLACE TABLE analytics.x CLONE audit.x`). A clone is fast and adds no storage until data changes. Clones are made table by table in dependency order, and a mart counts as published only when all of its tables have been cloned. Rollback uses BigQuery table snapshots and time travel. Cloud-specific calls (clone, snapshot, archive read) sit behind small helper functions in the executor, so only those change if the platform does. v4 Immediately before each clone, the publish step re-reads the dataset's publication state and any open outage breaker for its source, so a fix that finished after a breaker opened cannot publish.

Snowflake was considered. Its schema swap and zero-copy clones fit Write-Audit-Publish natively, but Snowflake on Google Cloud has no Indian region, and running it on AWS Mumbai would add cross-cloud data transfer and networking. It is not used in the pilot.

Healthcare requirements: Google's business associate agreement is signed for HIPAA and only covered services are used; every resource is pinned to `asia-south1` by an organisation policy for DPDP data residency; column masking and row access are enforced with BigQuery policy tags, dynamic data masking and row-level security; and every query is captured by Cloud Audit Logs. v4 Before the pilot, confirm in writing that the chosen model and agent runtime process requests in India, that both are covered by the business associate agreement, and that prompts are not used for model training. Until all three are confirmed, the agents stay disabled and every unknown case escalates to a human.

Infrastructure and deployment





Identical dev, test and prod environments, each built with Terraform. Code-fix sandbox (v2) is a separate project with synthetic data.

Figure 2. Deployment view: one environment, split into projects inside a single service perimeter.

Projects and environments. Dev, test and prod are separate projects, so a test run can never touch production data. Within each environment, data, the control plane and AI agents are separate projects, so agent access can be limited to masked views. Everything (datasets, buckets, IAM, network, alert policies) is built with Terraform and changed through reviewed pull requests, like rule changes.

Identity and permissions. Each component runs as its own service account through Workload Identity; no key files exist. Permissions enforce the publication gate, so it holds even if code is wrong.

Service account	Can	Cannot
Ingest	Write `raw`, the archive bucket and `ops.control_totals`	Read or write `analytics`
dbt build	Read `raw`; write `audit` and `quarantine`	Write `analytics`; use the reconciliation key
Value reconciliation v4	Read `raw` and `audit`; use the reconciliation KMS key to unwrap the keyset; write `evidence_restricted`	Write `raw`, `audit` or `analytics`; store values or digests
Publish	Clone tables from `audit` into `analytics`; read publication and breaker state	Anything else
Decision engine	Read test results, evidence and deployment metadata; write incidents in Cloud SQL	Change warehouse data
Recovery executor	Read the archive; write `audit` and `quarantine`; take locks	Write `analytics`
AI agents	Read masked views, logs and runbooks	Write anything; see unmasked PHI or `evidence_restricted`
CI/CD	Deploy through Workload Identity Federation	Hold long-lived keys

Area	Setting
Network	No public IPs (organisation policy); Cloud SQL on private IP; Private Google Access; VPC Service Controls around the data and agent projects; egress only through Cloud NAT to an allowlist (Slack, PagerDuty, GitHub); HA VPN or Cloud Interconnect to on-premises sources
Storage and encryption	Customer-managed keys in Cloud KMS for BigQuery, Cloud Storage and Cloud SQL; Bucket Lock and versioning on the archive; a separate, stricter bucket for evidence; a separate reconciliation key in `asia-south1`, usable only by the value-reconciliation identity v4
Availability and recovery	Cloud SQL high availability with point-in-time recovery; scheduled Airflow environment snapshots; locks expire after a lease (renewed while an action runs) so a crashed executor cannot block a partition; disaster recovery in `asia-south2` (Delhi) so data stays in India; recovery time and recovery point targets to be agreed
Audit logging	Cloud Audit Logs, including Data Access logs, for BigQuery, Cloud Storage and Cloud SQL, routed to a locked log bucket (and to Splunk if kept), with retention aligned to records policy
Fallback alerting	Cloud Monitoring uptime checks on the engine and executor page PagerDuty or email directly, independent of Slack; Pub/Sub dead-letter topics so no alert is silently lost
Cost guardrails	Maximum bytes billed on every dbt and engine query; custom BigQuery quotas per service account; budget alerts per project; on-demand pricing in the pilot, a slot reservation once volumes are known; token and time budgets for agent calls

Release process	Images built in CI and stored in Artifact Registry with vulnerability scanning (Binary Authorization optional); dev → test → prod promotion for dbt models, policy and infrastructure; the failure-injection suite runs in CI; every release is recorded as a GitHub deployment so the engine can read it v4

5. End-to-end flow

One flow covers the whole system. The numbers follow the order in which data and decisions actually move: a clean run goes straight from 1 to 7; a failure leaves the gate at 6, goes through 8 → 9 → 11 (or 12), and returns to 5 to be re-verified. Step 13 records every step.

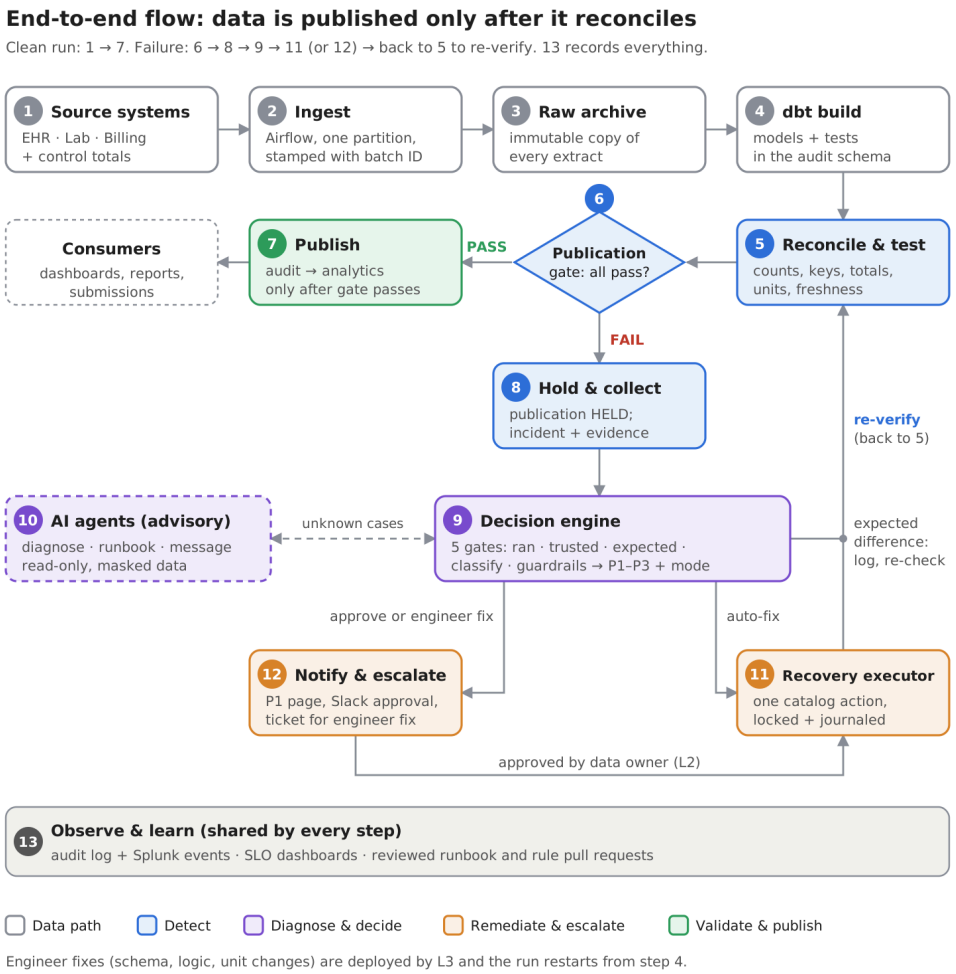


Figure 1. End-to-end flow. Colours show the stage: detect, diagnose and decide, remediate and escalate, validate and publish.

Step by step

1. **Source systems** (EHR, lab and billing) provide the data and its control totals per business day.
2. **Ingest** (Airflow) loads one partition idempotently, stamped with a batch ID.
3. **Raw archive** keeps an immutable copy of every extract, retained longer than the source keeps its records.
4. **dbt build** creates the models and runs the tests in the audit schema.
5. **Reconcile and test** checks the data against the source: counts, distinct keys, totals, referential integrity, freshness, schema, value ranges and units; and, for declared critical fields, record-level values v4.
6. **Publication gate** decides once per dataset and business date. If every blocking check passes and no incident holds the date, go to step 7; otherwise go to step 8.
7. **Publish** clones the verified audit tables into analytics. Consumers only ever see verified data. A clean run ends here.
8. **Hold and collect:** publication stays HELD, so consumers keep the last verified day, flagged as delayed. The collector turns errors and mismatches into signals and opens one incident with an evidence record. When several datasets from one source fail together, their incidents are grouped under one outage incident v4.

9. **Decision engine** runs five gates: did the checks run, is the check trustworthy, is the difference expected, which rule matches, and do the guardrails allow a fix. It reads the deployment context v4, sets severity (P1–P3) and the response mode: auto-fix, approve or engineer fix. Expected differences are logged and the checks re-run.
10. **AI agents** are called only for unknown or low-confidence cases. They diagnose the cause and suggest a fix; the engine re-checks the suggestion against the policy.
11. **Recovery executor** runs one catalog action under a lock, replaying from the raw archive where needed, and the flow returns to step 5 to re-verify.
12. **Notify and escalate:** P1 pages the on-call owner at once. Data-owner approvals (L2) go through Slack and, once approved, run through step 11. Engineer fixes (L3) get a ticket with evidence and, once deployed, the run restarts from step 4.
13. **Observe and learn** records every step to the audit log and Splunk, including auto-healed incidents, and proposes runbook, rule and threshold changes as reviewed pull requests.

How the four stages map to the steps

Stage	Steps	What happens	Example (13 October)
Detection	5, 6, 8	Reconcile against the source; the gate holds publication; an incident opens with evidence	800 encounters missing (10,000 vs 9,200)
Diagnosis	9, 10	Five gates classify the failure; agents explain only unknown cases	All 800 fall between 02:00 and 03:00 → `partial_extract`
Remediation	11, 12	One catalog action, or a Slack approval or an engineer fix	`reextract_from_raw` for 02:00–03:00
Validation	5 → 6 → 7	All checks re-run on the affected scope; publish only on a pass	10,000 vs 10,000 → published

Steps 9 to 11 run inside a governance boundary: least-privilege service identities, masked data for agents, and an audit record for every action.

6. Inputs and outputs

The system takes in data, metadata about the run, policy and human decisions, and produces trusted data, a publication decision and a complete record of what happened. The examples follow the 13 October run described in section 14.

Inputs: what the system takes in

Input	Where it comes from	Example
Source records	EHR, lab and billing systems (step 1)	Encounter row: `encounter_id E-10442`, `patient_id P-2231`, `admit_ts 2026-10-13 02:14 IST`, facility `BLR-01`
Control totals	Written by the extract to `ops.control_totals` in the same snapshot as its rows; declared as a dbt source	`claims | 2026-10-13 | batch b-20261013-01 | rows 10,000 | distinct claim IDs 10,000 | total ₹48,12,330.50`
Raw archive files	Object storage (step 3)	`raw/claims/load_date=2026-10-13/b-20261013-01.parquet`
dbt artifacts	`run_results.json`, `manifest.json`, `sources.json` (steps 4–5)	`{"unique_id": "test.claims.recon_claims", "status": "fail", "failures": 1}`
Stored test failures	dbt `--store-failures` tables	12 claim IDs in the stored failures of the raw-batch duplicate test (warning)
Orchestrator events	Airflow task state and errors	`extract_lab_results` failed: connection refused (attempt 1 of 3)
Logs	Splunk	`index=airflow dag_id=claims_pipeline status=failed earliest=-24h`

Run metadata	Replica lag, archive cutoff, patient index, orphan ages, prior attempts	Replica lag 12 s; archive cutoff 2026-10-06; patient P-2231 merged into P-1187
Dataset metadata	dbt `meta` tags	`criticality: high, owner: revenue-cycle, phi: true, source_system: billing, domain: revenue-cycle` v4
Critical-field declaration v4	dbt `meta` on models that keep one row per source record	`record_key: lab_result_id, critical_fields: [result_value, result_unit, collected_at]`
Deployment context v4	GitHub deployments, the Airflow DAG version and the dbt `manifest.json` hash	Release `r-2026.10.12-2`, commit `9f3c1ab`, deployed 2026-10-12 21:40 IST, changed models: `stg_lab_results`
Policy and action catalog	`policy.yml` in git, versioned	Policy 1.7.0: `never_auto: [schema_drift, fan_out, anomaly_or_unit, …, value_mismatch]`
Agent registry v4	Versioned agent configuration in git	`diagnosis@1.4`, pinned model version, prompt `diag-v7`, token and time budget
Runbooks	Git runbook repository	`runbooks/partial_extract.md`
Human decisions	Slack approvals and closed tickets	Data owner approves “remap P-2231 → P-1187”; ENG ticket closed with fix

Outputs: what the system produces

Output	Where it goes	Example
Published marts	Analytics schema → dashboards, reports, submissions (step 7)	`fct_daily_claims` for 13 Oct cloned into analytics at 05:40, verified
Publication state	Per dataset and business date	Claims 13 Oct: PUBLISHED. Lab results 20 Oct: HELD (unit change)
Delayed-data flag	Dashboards and reports	“Data delayed: last verified refresh 19 Oct 06:00”
Incident record with evidence	`ops` schema (step 8)	`encounters|2026-10-13|count_gap|02:00-03:00`, evidence `ev://2026-10-13/inc-0412`, severity P2, mode auto-fix
Outage group v4	`ops` schema (step 8)	`outage|lis|2026-10-13T01:00`, 4 child incidents, all HELD
Action journal	`ops` schema (step 11)	`reextract_from_raw`, window 02:00–03:00, attempt 1, recheck pass
Quarantined rows	Access-restricted `quarantine_*` tables	1 lab result with no parent anywhere after 24 hours
Mismatched keys v4	`evidence_restricted` dataset, behind an evidence reference	14 `lab_result_id` values with field name `result_value`; no values, no digests
Alerts and approval requests	Slack, PagerDuty (step 12)	“P1: lab results 20 Oct held, unit change (mmol/L). Evidence: `ev://2026-10-20/inc-0431`”
Tickets	Incident tool, for tracked engineering work only	Schema drift on `stg_lab_results`, with failing test IDs and evidence reference
Audit events and KPIs	Splunk and ops marts (step 13)	MTTR by P1/P2/P3, auto-recovery rate, stale-data duration, publication blocks, agent acceptance rate v4
Runbook, rule and threshold pull requests	Git, for human review	PR: “Add rule `lab_feed_token_expiry → retrigger_ingestion`”, recommend-only at first

Example: one incident in and out

In: the reconciliation test reports source 10,000 encounters vs target 9,200 for 13 October; the gap falls between 02:00 and 03:00; replica lag is 12 seconds; it is the first attempt; no release touched the encounter models in the last 24 hours.

Out: the incident record below, a replay of that hour from the raw archive, a passing re-check, a published mart and one line in the Slack summary.


```
{"run_date": "2026-10-13", "failure": "count_gap", "gap_rows": 800,
 "rule": "partial_extract", "action": "reextract_from_raw",
 "window": "02:00-03:00", "severity": "P2", "mode": "auto-fix",
 "attempt": 1, "policy_version": "1.7.0", "deployment_context": "none in 24 h",
 "outage_group": null, "recheck": "pass"}
```


7. Failures we must handle

Loud vs silent failures

Silent failures are the main reason this architecture exists: every task is green, but the data is wrong.

	Loud failures	Silent failures
What happens	The job crashes or errors	The job succeeds; the data is wrong
Examples	EHR replica timeout, SQL error, missing table, warehouse suspended	0 lab results loaded, duplicated claims, all-null column, join fan-out, stale feed, unit change, a value corrupted in transformation
Who sees it	The orchestrator, immediately	Nobody, until a report looks wrong
Caught by	Task status, `run_results.json`	Data checks: reconciliation, dbt tests, freshness, anomaly tests, value-level reconciliation
Typical fix	Retry with backoff	Targeted rebuild, quarantine, or escalate

Example. Loud: at 01:02 the EHR read replica restarts and the extract fails with *connection refused*; the orchestrator retries at 01:07 and succeeds. **Silent:** the lab interface returns an empty page because of an expired token; the extract succeeds with 0 lab results, dbt builds, every task is green, and only the volume check (0 rows vs a typical 18,000) reveals it.

Failure catalog

Failure	What it looks like	Caught by	Fix
Most likely (first month)			
Late-arriving records	Encounters closed or results filed just after midnight miss the load	Source vs target count per date	Wait, then reprocess the partition
Claim reversals missed	Reversals and re-adjudications after load are not picked up	Amounts differ, counts match	Incremental model on `updated_at` with a 3-day lookback
Time-zone cutoff	Source stores UTC; facilities report local time	Recurring gap in the early-morning hours	Convert in staging; reconcile on the same day definition
Duplicate claims or encounters	An ingestion retry loads a batch twice	Duplicate-key test on the raw batch (warning)	Removed by the staging model on every build; logged
Partial extract	One hour of encounters missing	Hourly count comparison	Replay the window from the raw archive

Likely over time			
Schema drift	A source column added, renamed or retyped	`not_null`, or dbt column error	Escalate (engineer fix)
Join fan-out	Joining encounters to diagnoses duplicates encounters	Fact rows vs distinct keys	Fix the model grain; escalate
Stale feed	Lab or ADT interface paused	`dbt source freshness`	Re-trigger ingestion; escalate after N tries
Unit change	Glucose arrives in mmol/L instead of mg/dL; amounts in paise	Value-range and anomaly tests	Block publish, escalate
Late change beyond lookback	A claim is reversed or re-adjudicated weeks after load	Weekly 90-day reconciliation by amount and distinct keys	Targeted rebuild of the affected dates
Upstream dataset held	A mart joins claims (held) with encounters (published)	Dependency check at the publication gate	Hold the dependent mart
Test patients and soft deletes	Test or voided records counted	`accepted_values`, shared filter	Filter in staging on both sides
Correlated upstream outage v4	Several datasets from one source fail in the same run: the lab interface is down, so lab results, panels and orders all show count gaps	Upstream-outage breaker: failures grouped by `source_system` within a time window	Hold every affected dataset, no per-dataset fixes, one grouped incident; re-evaluate each dataset once the source recovers
Critical-value mismatch v4	Counts, keys and totals match, but a critical field changed between the raw batch and the model: a decimal truncated, a timestamp shifted, a code remapped	Value-level reconciliation on declared critical fields	Block publish, escalate (engineer fix); evidence lists mismatched keys and field names only
Suspected deployment regression v4	A model fails, or starts producing mismatches, soon after a release changed it or its upstream	Deployment context in the evidence record, when no data rule explains the failure	Engineer fix with the release attached; rollback is an L3 decision

Reconciliation-phase failures

The check itself can be wrong: it raises false alarms or, worse, passes when data is wrong.

Group	Failure	What happens	Fix
Moving source	Moving target	The EHR keeps changing while it is counted	Capture control totals inside the extract's own consistent snapshot and compare against those, not against the live source (section 11)
Moving source	Replica lag	Replica is behind, so records look missing	Check lag first; wait or read from primary
False matches	Offsetting errors	50 missing + 50 duplicates: counts match	Compare count, distinct keys and totals together
False matches	Same bug both sides	Source query and model share a wrong filter	Write reconciliation SQL independently of model logic
Formatting	Precision, NULLs, case	`numeric(12,4)` vs `(12,2)`; NULL vs 0; code-set casing	Normalize in staging; small tolerance
Check didn't run	Silent skip	Test excluded by a selector; green means nothing ran	Assert the expected test count
Check didn't run	Missing control totals	The extract wrote no totals for the batch, so the reconciliation test compares nothing and passes	Assert exactly one control-totals row per dataset, business date and batch (blocking)
Noise	Flapping, thresholds	Pass and fail alternate; tiny gaps page every night	Two consecutive failures for low-risk checks only; warn above 0.1%, block above 1%

Correlated noise v4	One outage, many alerts	A source outage makes every dataset from it fail, each with its own incident and fix	Upstream-outage breaker groups them, holds them and suppresses per-dataset fixes (section 9)

v4 Value-level reconciliation compares the raw batch with the warehouse model, so it proves the pipeline preserved each value. It does not prove the source value is right: a unit switched at the source arrives identical on both sides, and value-range and anomaly tests remain the control for that.

> **Example: offsetting errors.** On 11 October the claims count passes (10,000 vs 10,000), but a retried batch loaded 50 claims twice and a partial extract dropped 50 others. Distinct claim IDs (9,950) and billed totals both fail. The engine treats it as duplicates plus missing records: it replays the gap from the raw archive, and the rebuild that follows deduplicates the retried batch; then all three metrics are re-checked.

8. Archive and orphaned records

Archiving and patient merges are the main reason a correct warehouse can look wrong. Once the source has archived a period, it is no longer the reference for it: the raw archive and the warehouse take over that role.

Situation	Healthcare example	Rule
Warehouse holds history the source has archived	Encounters older than the retention window move to the archive store	Reconcile only the active window (for example, the last 7 days); older differences are logged as expected
Child arrives after its parent was archived	A claim adjustment in October references a June encounter	Check parents against the warehouse dimension and raw archive, not the live source; parent found → link
Patient records merged	Lab results still point to a retired duplicate patient ID	Remap to the surviving ID from the patient index (`remap_merged_id`)
Parent not yet loaded	A lab result arrives before its encounter is registered	Link to an unknown member and recheck next run; most resolve on their own
Parent missing everywhere after 24 h	Payment for an encounter that exists in no system	Quarantine (access-restricted) and escalate
Archive deletes reaching the warehouse	A sync mirrors a purge, orphaning claims and payments	Never propagate source deletes; `full_refresh: false` on clinical and claims models

Because quarantined rows are held outside the marts, the pass rule is **source total = mart total + quarantined total**. This holds only because quarantine moves records rather than copying them: the mart and the quarantine table are mutually exclusive populations at the same grain (one row per claim or result), enforced by a uniqueness test across both, so no record is counted twice. The raw archive is also the recovery tool: a defect found months later is fixed by replaying archived extracts, because the source no longer has them.

> **Example.** On 14 October the EHR's retention job archives 1,200 encounters older than its window. A naive full-history check reports 1,200 "extra" warehouse rows; the engine sees the dates are before the cutoff and the IDs are in the archive index, and logs them as expected. The same night, a claim adjustment for a June encounter arrives: its parent is found in the warehouse dimension, so it links normally instead of becoming an orphan.

9. Decision engine

The engine auto-fixes failures that are well understood and safe to repeat, and escalates everything else with evidence. It runs after every dbt build and reads `run_results.json`, stored failure tables, reconciliation results and run metadata (replica lag, archive cutoff, patient-index merges, orphan ages, prior attempts), plus the deployment context and any open outage group for the dataset's source v4.

Five gates

1. **Did the checks run?** Missing tests, missing control totals or an unreachable source → block publish, escalate.

2. **Is the check trustworthy?** Replica lag, moving source or a flapping low-risk check → wait and recheck, don't fix. Never applied to P1 or PHI incidents. v4 If an outage group is open for the dataset's source, the check is not trustworthy: the engine holds and waits, and does not fix.
3. **Is it expected?** Archive gaps, orphans with archived or merged parents, late records, gaps within tolerance → log and continue.
4. **Classify and pick a fix.** Match symptoms to a known rule; unrecognised failures always escalate. v4 The engine reads the deployment context. If a data rule matches, the decision is unchanged and the context is attached. If no data rule matches and a recent release touched the failing models or their upstream, the incident is classified `suspected_regression` (engineer fix). If a data fix later fails verification while a release is flagged, the incident escalates as a suspected regression.
5. **Guardrails.** Fix already tried at the cap, unsafe for this type, or in the never-auto list → escalate. v4 While a source's breaker is open, automatic actions for that source become recommendations; while the global breaker is open, every automatic action does.

Key rules

- **Trust before diagnosis:** if the source replica is lagging or a low-risk check is flapping, the engine waits and rechecks instead of fixing. If expected tests did not run, it holds publication and escalates.
- **Immediate escalation overrides confirmation:** the two-run confirmation window applies only to transient, low-risk failures. P1 incidents, security or PHI exposure, unsafe publication and checks that did not run escalate at once. Unknown failures hold publication where published data could be affected and escalate without inventing a fix.
- **Expected before failure:** archive gaps, orphans whose parent is archived or merged, records younger than the late-arrival window and gaps within tolerance are logged as expected differences.
- **Rule priority:** security or PHI exposure > logic error > schema drift > data issue > transient.
- The gate auto-runs an action only when **confidence is high, blast radius is low, the action is reversible and attempts are below the cap (2)**. `full_refresh` is never automatic.
- Every decision stores its policy version, so any incident can be replayed.
- **A deployment is a clue, not a cause** v4 : timing alone never assigns blame and never triggers a rollback.
- **Three kinds of repetition, three responses** v4 : retries within one run stop at `max_attempts_per_action`; the same failure across runs feeds the recurrence count and the learning loop; many datasets failing together is a shared outage, handled by the breaker.
- **Scope follows the cause** v4 : a single-dataset failure holds that dataset and its dependants only; a source outage holds that source's datasets; only the global breaker affects every source, and even then it switches automation to recommend-only while healthy sources keep publishing.

Every failure mapped to a decision

Failure	Signal	Decision	Mode
Source timeout	Extract task error	Retry with backoff	Auto-fix
dbt compile or SQL error	`run_results` status error	Escalate	Engineer fix
Partial extract	Count gap in one time window	Replay window from raw archive, rebuild	Auto-fix
Duplicate claims or encounters	Duplicates in the raw batch, same batch ID	Removed by the staging model on rebuild; logged	Auto (built into the model)
Duplicates in a staging model	`unique` fails on `stg_claims`	Block publish, escalate (dedupe logic broken)	Engineer fix
Claim reversals not applied	Amounts differ, counts match	Lookback rebuild	Auto-fix
Late documentation or results	Small gap near the day boundary	Wait, re-check	Auto-fix
Stale feed	Freshness test fails	Re-trigger ingestion, then escalate	Auto-fix, then engineer
Archived history	Warehouse exceeds source before the cutoff	Log as expected	Auto (no fix)

Orphan, parent archived	Parent found in archive or warehouse	Link, log as expected	Auto (no fix)
Orphan, parent not yet loaded	Orphan under 24 hours old	Unknown member, recheck	Auto-fix
Orphan, patient merged	Patient ID retired in the index	Remap to surviving ID	Approve
Orphan over 24 h, no parent	Past the age limit	Quarantine, escalate	Engineer fix
Unit change	Value-range or anomaly test	Block publish, escalate	Engineer fix
Join fan-out	Fact rows exceed distinct keys	Block publish, escalate	Engineer fix
Schema drift	Missing or retyped column	Block publish, escalate	Engineer fix
Checks did not run	Fewer tests than expected, or no control-totals row for the batch	Block publish, escalate	Engineer fix
Offsetting errors	Counts match; keys or totals don't	Replay gap; the rebuild deduplicates	Auto-fix
Late change beyond lookback	Weekly 90-day reconciliation finds an amount gap	Rebuild the affected dates	Auto-fix
Upstream dataset held	A dependency of the mart is HELD	Hold the dependent mart	Auto (no fix)
Correlated upstream outage v4	Breaker threshold reached for one source	Hold the source's datasets, group incidents, one alert	Auto (no fix), then each child re-evaluated
Critical-value mismatch v4	Value-level test above its block level	Block publish, escalate	Engineer fix
Suspected deployment regression v4	No data rule matched; a recent release touched the scope	Hold, recommendation with the release attached	Engineer fix

Roughly two-thirds of these are handled automatically; the rest are code, schema or business-logic problems, for which the engine blocks bad data and hands over evidence.

Rules as configuration

Rules live in `policy.yml` and are evaluated in three fixed stages, so no ordinary rule can override a safety requirement:

1. **Safety overrides first.** Mandatory escalations, never-auto failures and the outage breaker v4 are checked before any other rule. A wait, tolerance or expected-difference rule can never cancel them.
2. **Ordinary rules.** Walked top to bottom, first match wins, for everything the safety stage did not settle.
3. **Global publication gate last.** Evaluated once per dataset and business date, across all of its incidents. One passing check or one expected difference can never authorise publication while another incident still holds it.

When engineers resolve the same escalation repeatedly, it becomes a new rule through the learning approval boundary (section 12). The full policy for version 1.7.0, with the v4 entries in place:

```
policy_version: 1.7.0

safety:                       # stage 1: evaluated first, cannot be overridden
  immediate_escalation: [p1, phi_exposure, unsafe_publication, checks_missing, unknown]
  never_auto: [schema_drift, fan_out, anomaly_or_unit, compile_error, full_refresh,
               value_mismatch]                          # v4: value_mismatch added
  confirmation_runs: 2        # transient, low-risk only
  max_attempts_per_action: 2
  upstream_outage_breaker:                              # v4
    group_by: source_system   # dbt meta tag on sources and models
    window_minutes: 30        # failures this close together are grouped
    inventory: manifest_datasets_by_source_system       # datasets scheduled for this run
    open_when:
      min_datasets_failing: 3
      or_min_share_of_source_failing_percent: 50        # failing ÷ inventory, in percent
    if_inventory_unknown: count_rule_only               # share rule off; logged as a warning
    counted_failures: [extract_error, count_gap, stale_feed, control_totals_missing]
    on_open:
      action: hold_publish    # all datasets of that source, plus dependants
      suppress: [auto_fix]    # per-dataset fixes become recommendations
```

```
      incident: grouped       # one parent incident; children linked and HELD
      severity_floor: P2      # P1 when published data is affected
    close_when_all: [source_probe_passed, freshness_passed]   # probe alone keeps it open
    on_close: reevaluate_children                       # each child re-runs the five gates
    global:
      open_when: { min_sources_failing: 2 }
      on_open: { mode: recommend_only, notify: [platform_oncall] }  # healthy sources still publish

evidence:                                               # v4
  deployment_context:
    sources: [github_deployments, airflow_dag_version, dbt_manifest_hash]
    lookback_hours: 24
    match_scope: failing_models_and_upstream            # resolved through dbt lineage
    effect:
      when_data_rule_matches: attach_only               # decision unchanged
      when_no_data_rule_matches: suspected_regression   # engineer fix
      when_fix_fails_verification: escalate_as_suspected_regression
    never: [auto_rollback]

value_level_reconciliation:                             # v4, design in section 11
  enabled_for: models_with_meta_critical_fields
  compare: raw_batch_vs_source_grain_model
  method: keyed_digest        # deterministic encryption, not HMAC
  keyed_digest:
    implementation: bigquery_deterministic_aead
    kms_key_uri: >-
      gcp-kms://projects/<data-project>/locations/asia-south1/keyRings/recon/cryptoKeys/recon-digest
    wrapped_keyset_secret: recon-wrapped-keyset         # Secret Manager; recon identity only
    record_versions: [keyset, kms_key]
  canonicalization: macros/recon/canonical_value.sql    # one implementation, both sides
  policy_outcome: threshold   # the engine decides from the counts below
  outcome:                    # defaults; datasets override in dbt meta
    p1_datasets: { block_if_mismatched_rows_gt: 0 }     # any single mismatch blocks
    other:
      warn_if_mismatched_percent_gt: 0.01               # percent of compared rows (0.01 %)
      block_if_mismatched_percent_gt: 0.1               # percent of compared rows (0.1 %)
  evidence:
    store: [record_key, field_name, key_version]
    never_store: [values, digests]
    location: evidence_restricted

rules:                        # stage 2: first match wins
  - name: checks_missing
    when: { tests_ran_lt_expected: true }
    action: block_and_escalate
  - name: control_totals_missing
    when: { control_totals_rows_ne: 1 }
    action: block_and_escalate
  - name: unstable_source
    when: { replica_lag_minutes_gt: 5 }
    action: wait_and_recheck
  - name: archived_gap
    when: { failure: count_gap, date_before_archive_cutoff: true }
    action: log_expected
  - name: orphan_parent_archived
    when: { failure: relationships, parent_in_archive_or_warehouse: true }
    action: link_and_log
  - name: orphan_patient_merged
    when: { failure: relationships, patient_id_retired_in_mpi: true }
    action: remap_merged_id   # mode: approve
  - name: young_orphans
    when: { failure: relationships, max_age_hours_lt: 24 }
    action: link_unknown_member
  - name: within_tolerance
    when: { failure: count_gap, gap_percent_lt: 0.1 }   # percent (0.1 %); v4 renamed for clarity
    action: log_and_publish
  - name: raw_duplicates
    when: { failure: raw_duplicate }
    action: log_and_continue  # the staging model already deduplicates
  - name: duplicates_in_model
    when: { failure: unique }
```

```
    action: block_and_escalate # dedupe logic broken: engineer fix
  - name: partial_extract
    when: { failure: count_gap, gap_in_single_window: true }
    action: reextract_from_raw
  - name: late_change_outside_window
    when: { failure: amount_gap, date_outside_lookback: true }
    action: rebuild_partition # each affected date
  - name: upstream_held
    when: { failure: upstream_not_published }
    action: hold_publish
  - name: anomaly
    when: { failure: anomaly_or_unit }
    action: block_and_escalate
  - name: value_mismatch                              # v4
    when: { failure: value_mismatch }
    action: block_and_escalate
  - name: suspected_regression                        # v4
    when: { deployment_touched_scope: true, no_data_rule_matched: true }
    action: open_recommendation   # engineer fix, release attached; publication stays held
  - name: fallback
    when: {}
    action: escalate

publication_gate:             # stage 3: once per dataset and date
  publish_only_if: [all_blocking_tests_passed, no_open_incidents_holding,
                    reconciliation_passed, all_upstream_published,
                    no_open_source_breaker]             # v4
  caveat_only_if: [dataset_allows_caveat, gap_within_declared_limit,
                   no_never_auto_failure, owner_approved]
```

Definitions v4

- **`block_and_escalate`** is a decision outcome, not an executable action: it runs the existing `hold_publish` and `open_recommendation` actions with mode engineer fix. v4 adds no catalog actions.
- **Inventory** is the set of datasets tagged with that `source_system` in the dbt manifest and scheduled for this run. If it cannot be read, only the count rule applies and the gap is logged.
- **Publish-time check:** the publish step re-reads the dataset's publication state and any open breaker for its source from Cloud SQL immediately before each clone. A fix that finishes after its source's breaker opened therefore cannot publish; its result waits for the breaker to close and the five gates to run again. The action lock alone does not give this guarantee.
- **Thresholds** named `_percent` are percentages of compared rows; every value above is an example, changed only through the reviewed pull-request process.

> **Example: one failure through the five gates.** On 13 October the encounter count reports 800 missing.
>
> 1. **Did the checks run?** Yes: 42 of 42 expected tests executed, and the control-totals row for the batch is present.
> 2. **Trustworthy?** Replica lag is 12 seconds, the comparison used the raw archive file, and no outage group is open for the EHR source, so yes.
> 3. **Expected?** No: the date is inside the active window and the gap is 8%, far above tolerance.
> 4. **Classify.** All 800 encounters fall between 02:00 and 03:00 → `partial_extract` → `reextract_from_raw`. No release touched the encounter models in the last 24 hours.
> 5. **Guardrails.** First attempt, and `count_gap` is not never-auto → proceed.

Evidence contract

A successful job is never taken as a successful reconciliation. Every incident carries the same evidence record, and the engine refuses to decide on an incomplete one.

Field	Content	Example

Incident key	Stable key: dataset, business date, failure type, scope	`encounters|2026-10-13|count_gap|02:00-03:00`
Snapshot and business date	Batch ID of the extract snapshot, the consistency strategy used (control totals or raw-file fallback), and the business-day definition	batch `b-20261013-01`, repeatable-read snapshot; day = facility local time
Expected vs actual	Counts, distinct keys and totals, source and target	10,000 vs 9,200 encounters; claims ₹ totals
Failed checks and scope	Failed test IDs, affected models and partitions	`test.claims.recon_claims`; `stg_encounters`, 13 Oct
Completeness	Tests expected vs executed, control-totals row present, and evidence timestamps	42 of 42 executed at 01:18; control totals present
Evidence reference	Pointer to stored failing rows in the protected evidence store, never the rows themselves	`ev://2026-10-13/inc-0412`
Outage group v4	Parent incident key when the breaker grouped this incident; empty otherwise	`outage|lis|2026-10-13T01:00`
Deployment context v4	Releases within the lookback window that touched the failing scope or its upstream; a clue, never proof	`9f3c1ab` changed `stg_lab_results` 3 h before the run
Value-level result v4	Fields compared, mismatched record count per field, keyset and KMS key versions, and an evidence reference to the mismatched keys	`result_value`: 14 of 18,020; keyset v3; `ev://2026-10-13/inc-0419/keys`
Agent provenance v4	For every agent call: agent and version, model and version, prompt version, input evidence references, output, validation result	`diagnosis@1.4`, prompt `diag-v7` → `partial_extract` (0.86), valid

Never in the evidence record: raw clinical values, plain or keyed digests of them, or patient identifiers outside the restricted evidence store.

Financial totals: amounts are compared after rounding both sides to 2 decimal places, with a tolerance of ₹1 per business day. Null amounts are counted separately rather than treated as zero, so a run of nulls cannot hide inside a matching total.

Incident lifecycle

Two independent fields are tracked. `lifecycle_state` belongs to the incident and says where its handling stands; `publication_state` belongs to the dataset and business date and says whether consumers may see it. An incident is marked resolved only after verification passes.

DETECTED → TRIAGED → REMEDIATING → VERIFYING → RESOLVED

State	Meaning	Leaves when
DETECTED	Signal collected, incident key and evidence record created	Evidence complete → TRIAGED
TRIAGED	Gates run, severity and mode decided	Auto-fix → REMEDIATING; approval → AWAITING_APPROVAL; expected difference → RESOLVED (expected); engineer fix, mandatory escalation or no safe action → ESCALATED; grouped under an open outage → stays TRIAGED and HELD until the group closes v4
AWAITING_APPROVAL	Data owner asked in Slack	Approved → REMEDIATING; rejected or timed out → ESCALATED
REMEDIATING	One catalog action running under lock, attempt journaled	Action finished → VERIFYING; budget exhausted → ESCALATED
VERIFYING	All checks re-run on the affected scope	Pass → RESOLVED and publish; fail → REMEDIATING (within budget) or ESCALATED
ESCALATED	Human-owned, with evidence; ticket only if the escalation policy requires one	Human fix deployed → VERIFYING

RESOLVED	Verification passed, or difference confirmed as expected	Final; feeds the learning loop

When the retry budget expires, automatic remediation stops and the incident moves to ESCALATED with its evidence and attempt history.

Publication state	Meaning	Set when
PENDING	New data built in audit, not yet checked	dbt build finishes
HELD	Publication blocked; consumers keep the last verified day, flagged as delayed	Any blocking check fails, any open incident for this dataset and date is not yet resolved, a dataset it depends on is held, or its source's outage breaker is open v4
PUBLISHED	Audit tables cloned into analytics	The global publication gate passes
PUBLISHED_WITH_CAVEAT	Published with a visible caveat on dashboards, for a known small gap	The dataset allows caveats, the gap is within its declared limit, no never-auto failure is open, and the data owner approves

For example, a unit change is reported as `lifecycle_state = ESCALATED` and `publication_state = HELD`.

Publishing with a caveat. Holding is the default, but it is not always the best business outcome: for the 06:00 bed meeting, a known gap of a few records may be more useful than yesterday's numbers. Each dataset declares in its dbt `meta` whether a caveat is allowed, the largest gap allowed and the approver. A caveat is never possible for a never-auto failure (unit change, schema drift, fan-out, value mismatch), for PHI exposure, or for a dataset that has not declared it.

Severity, support levels and response modes

The response mode sets *how* an incident is handled; the severity sets *how fast*. Severity comes from dataset criticality (a dbt `meta` tag, with clinical and regulatory datasets ranked highest), whether published data is affected, blast radius and time stale. The modes map onto the team's support levels: the engine acts as L1 (auto-fix, within the 30-minute L1 budget), the data owner or data engineer is L2 (approvals and investigation), and the development team is L3 (engineer fixes).

Severity	Rule	Engine action (L1, within 30 min)	Owner action (L2/L3, no agreed target yet)
P1	Clinical, financial or regulatory data that is already published is wrong or missing, or the blast radius is wide	Hold publish; run a safe fix in parallel	Data owner and on-call engineer paged at once
P2	Important data delayed or partly wrong; a workaround exists	Auto-fix within 30 min, then escalate	Approve the proposed fix or take it as an engineer fix
P3	Low impact, caught before publication	Auto-fix; escalate after 30 min if it fails	Review in the normal backlog

Severity rises when data stays stale past its freshness SLA, so no incident waits unseen. L2 and L3 have no agreed response target yet; this does not mean incidents can sit unattended, because severity and notification rules keep urgent incidents visible. Targets will be agreed with the SRE and support process owners.

Escalation policy: alerts, approvals and tracked work

Escalation is mandatory whenever the policy requires it; a ticket is only one way of managing it. Urgent alerts, human approvals and tracked engineering work are kept distinct, so a ticket is created only when work must be owned and tracked to closure.

Situation	Response	Ticket needed?
Critical production issue, such as wrong published lab results or claims totals	Page the on-call engineer and data owner; hold publication	Not necessarily immediately

Safe automatic recovery succeeds	Record the fix in the audit log and include it in the Slack summary	No
Data-owner approval required, such as remapping a merged patient ID	Send a Slack approval request and log the decision	Not necessarily
Orphan unresolved after 24 hours	Quarantine the record and escalate with evidence	Recommended
Schema drift, dbt compile error, join fan-out, unit change or value mismatch	Hold publication and hand over evidence for an engineering fix	Recommended
Shared upstream outage v4	One grouped alert naming the source, the held datasets and the downstream reports; hold publication for that source	Recommended, for the source-system team
Recurring upstream failure, such as a lab interface that keeps stalling	Notify the source-system team and track root-cause resolution	Recommended

10. Remediation playbook

Each action is a small, idempotent procedure the executor runs. Actions that change data in the warehouse run through dbt; actions that control timing, publication or people run in the orchestrator. The **Runs in** column shows which. Running any action twice gives the same result.

Action	Runs in	What it does	How
`retry_task` / `dbt_retry`	Orchestrator + dbt	Rerun the failed step	Airflow retry with backoff; `dbt retry`
`wait_and_recheck`	Orchestrator	Let late records or replicas catch up	Reschedule checks in 15–30 minutes
`reextract_from_raw`	Orchestrator, then dbt	Reload a missing window	Airflow replays the window from the raw archive (or re-extracts into it); dbt rebuilds with `dbt build --select stg_encounters+ --vars '{"reprocess": true}'`
`rebuild_partition`	dbt	Rebuild one day	Incremental model with the `insert_overwrite` strategy, replacing that date's partition in one statement; also used for dates found by the weekly reconciliation
`lookback_rebuild`	dbt	Pick up reversals and late changes	Incremental model reprocesses the last N days by `updated_at`
`link_unknown_member`	dbt	Keep orphans in totals while parents arrive	`coalesce(encounter_key, -1)`; re-links on the next build
`remap_merged_id`	Orchestrator, then dbt	Point records at the surviving patient	Owner approves in Slack; dbt joins to the patient-index merge table
`quarantine_rows`	dbt	Park records that can't be trusted	Access-restricted `quarantine_*` model; the mart excludes those rows; counted in reconciliation
`log_expected`	Orchestrator	Record expected differences	Engine writes to `expected_differences`; no data change
`hold_publish`	Orchestrator	Keep bad data from users	Skip the publish clone; consumers keep the last verified day
`open_recommendation`	Orchestrator	Hand over to a human	Slack approval or engineer ticket with evidence and attempts

There is no separate dedupe action: duplicates are removed by the staging model itself on every build (see section 11), so every rebuild the engine triggers also deduplicates, and nothing changes a dbt-managed table outside dbt.

Order when several failures happen at once

1. Blockers first: run safe fixes, but don't publish if anything blocks.
2. Re-extract missing partitions and parents from the raw archive.

3. Rebuild affected models once, selecting the model and everything downstream. The rebuild also deduplicates.
4. Link or remap orphans last; re-extraction may have resolved some.
5. Re-run all checks, not only the failed ones, since fixes can cause new problems.

Shared upstream outage v4

A shared outage is handled with existing actions; v4 adds no catalog action.

When a source's breaker opens

1. Every dataset of that source, and every mart that depends on one, is set to HELD with `hold_publish`. Consumers keep the last verified day, flagged as delayed.
2. Child incidents are linked to one parent incident (the outage group). Fixes queued for the children are cancelled, not run.
3. A fix already running finishes under its lock and is journaled, but its output cannot publish: the publish step re-checks the publication state and open breakers just before cloning (section 9).
4. The Communication agent sends one alert for the group, naming the source, the affected datasets and the downstream reports. A P1 pages once for the group, not once per dataset.
5. Each held dataset keeps its own evidence record, so nothing is lost when the group closes.

When it closes

1. The breaker closes only when the source probe and the freshness check both pass. If the probe passes but freshness still fails, the breaker stays open and the datasets stay held. A timer alone never closes it.
2. The engine re-runs the five gates for each child, oldest business date first. Each child then follows its normal path: auto-fix, approve or engineer fix.
3. A child whose retry budget was used before the outage keeps that count; the outage does not reset it.

Other sources keep running. One source's breaker never stops safe recovery or publication for another source. Only the global breaker switches all automation to recommend-only, and healthy sources keep publishing.

11. Role of dbt

dbt is not the decision engine. It does not watch, wait, retry on a timer or choose between fixes; that logic lives in the orchestrator. dbt is the eyes and hands inside the warehouse: every fix that changes warehouse data runs as a dbt build, while the orchestrator owns timing, publication and people.

dbt does	How
Transformations	Incremental models such as `stg_encounters` and `stg_claims`, so a single day can be rebuilt cheaply; deduplication is built into the staging models
Reconciliation checks	Singular tests comparing source and target; built-in `unique`, `not_null`, `relationships`, `accepted_values`; `dbt source freshness`; packages such as `dbt-utils`, `dbt-expectations`, `audit_helper` and Elementary; value-level comparison of critical fields v4
Evidence for decisions	`run_results.json` lists what passed or failed; `--store-failures` writes failing rows to tables the engine can query
Remediation	`dbt build --select model+ --vars ...` for partition rebuilds, replays and lookbacks; `dbt retry` to resume from the point of failure
Classification metadata	`meta` tags carry dataset criticality, owner, PHI flags, source system and critical fields v4 for the engine
History and audit	Snapshots show a figure before and after a fix; `on-run-end` hooks write to the audit table

Reconciliation test. Filtering both sides by an `updated_at` watermark is not enough: a claim updated after the watermark drops out of the source count while its earlier version is still in the warehouse. Instead, the extract runs in one read-only, repeatable-read transaction and writes the source control totals for that same snapshot to `ops.control_totals`, tagged with its batch ID. dbt reads them as a declared source, so they appear in lineage and get a freshness check of their own. The test compares the warehouse, as of that batch, against those totals, so both sides describe exactly the same point in time.

```
# models/staging/_sources.yml
sources:
  - name: ops
    schema: ops
    tables:
      - name: control_totals
        description: Source control totals written by the extract, in the same snapshot as its rows.
        loaded_at_field: captured_at
        freshness:
          error_after: { count: 6, period: hour }
```

```
-- tests/recon_claims.sql
-- Source side: totals captured by the extract in the same snapshot as its rows.
with src as (
  select row_count c, distinct_keys k, null_amounts n, total_amount amt
  from {{ source('ops', 'control_totals') }}
  where dataset = 'claims'
    and business_date = '{{ var("run_date") }}'
    and batch_id = '{{ var("batch_id") }}'),
-- Target side: the warehouse state as of that batch (batch IDs are monotonic).
tgt as (
  select count(*) c, count(distinct claim_id) k,
         countif(billed_amount is null) n,
         round(sum(billed_amount), 2) amt
  from {{ ref('stg_claims') }}
  where service_date_local = '{{ var("run_date") }}'
    and _batch_id <= '{{ var("batch_id") }}')
select * from src, tgt
where src.c != tgt.c or src.k != tgt.k or src.n != tgt.n
   or abs(src.amt - tgt.amt) > 1          -- ₹1 tolerance after rounding
```

If no control-totals row exists for the batch, `from src, tgt` returns nothing and the test above would pass silently. A separate blocking test therefore asserts that exactly one row exists for each dataset, business date and batch:

```
-- tests/control_totals_present.sql     (policy_outcome: blocking)
{{ config(meta={'policy_outcome': 'blocking'}) }}
select '{{ var("batch_id") }}' as batch_id
where (select count(*) from {{ source('ops', 'control_totals') }}
       where dataset = 'claims'
         and business_date = '{{ var("run_date") }}'
         and batch_id = '{{ var("batch_id") }}') != 1
```

If a source system cannot provide a consistent snapshot (for example, a vendor API), the fallback is to reconcile against the raw archive file of the batch itself, and to record in the evidence contract which strategy was used.

Example: deduplication in the staging model. `stg_claims` keeps exactly one row per claim using `row_number()`, ordered by load time with a stable tie-breaker (`_row_id`, a unique ID assigned at ingestion), so two copies with the same load time can never both survive. Rows with a null claim ID pass through untouched: the `not_null` test catches them and they are quarantined. Because the rule lives in the model, every build, including every rebuild the engine triggers, produces deduplicated data; nothing changes the table outside dbt. The same model carries the 3-day lookback that picks up claim reversals and is never fully refreshed. Deduplicating inside the model also keeps the `merge` safe, since a merge fails or behaves unpredictably when the incoming batch holds two rows with the same key. On 13 October the rebuild dropped 12 duplicate claims before publish.

```
-- models/staging/stg_claims.sql
{{ config(materialized='incremental', incremental_strategy='merge',
          unique_key='claim_id', full_refresh=false,
          partition_by={'field': 'service_date_local', 'data_type': 'date'},
          meta={'criticality': 'high', 'owner': 'revenue-cycle', 'phi': true}) }}

with src as (
    select * from {{ source('billing', 'claims') }}
    {% if is_incremental() %}
    where date(updated_at) >= date_sub(date '{{ var("run_date") }}', interval 3 day)
    {% endif %}
),
ranked as (
    select *,
           row_number() over (
               partition by claim_id
               order by _loaded_at desc nulls last, _batch_id desc, _row_id desc
           ) as _dup_rank
    from src
)
select * from ranked
where claim_id is null or _dup_rank = 1
```

Duplicates are still detected, at the raw layer, so the incident record and the Slack summary show how many were removed. The `unique` test on `stg_claims` remains as a safety net: if it ever fails, the dedupe logic itself is broken, which is an engineer fix.

```
-- tests/raw_claims_duplicates.sql      (policy_outcome: warning)
{{ config(severity='warn', store_failures=true, meta={'policy_outcome': 'warning'}) }}
select claim_id, count(*) as copies
from {{ source('billing', 'claims') }}
where _batch_id = '{{ var("batch_id") }}'
group by claim_id
having count(*) > 1
```

Relationship test against warehouse history (so archived parents are not orphans):

```
models:
  - name: stg_lab_results
    columns:
      - name: encounter_id
        tests:
          - relationships:
              to: ref('dim_encounters')      # keeps archived history
              field: encounter_id
              config:
                store_failures: true
                severity: warn               # dbt exit code only
                meta: { policy_outcome: blocking_unless_explained }
```

Test outcomes are decided by policy, not by dbt severity. dbt's `warn` or `error` only sets dbt's exit code. Every test also carries a `policy_outcome` in its `meta`, and the publication gate reads that:

policy_outcome	Effect on publication	Example
informational	Logged only	Row-count trend
warning	Logged and reported in the summary; never blocks	Orphans under 24 hours linked to an unknown member; duplicates in the raw batch
blocking_unless_explained	Blocks unless every failing row is classified as expected or has been quarantined	Relationship test on lab results
blocking	Always blocks publication	Reconciliation, control totals present, unique, unit and grain tests
threshold v4	The test returns every failing row; the engine compares the count with the dataset's thresholds and treats it as blocking above the block level, warning between the warn and block levels, and informational below. For P1 datasets the block level is zero, so one mismatch blocks	Value-level reconciliation

> **Example: what dbt hands the engine.** After the 13 October run, `run_results.json` contains `{"unique_id": "test.claims.recon_claims", "status": "fail", "failures": 1}`, and the stored failure table holds the numbers (source 10,000, target 9,200). The engine decides "partial extract" and calls dbt back with `--vars '{"run_date": "2026-10-13", "reprocess": true}'`. dbt never decides anything; it only reports and executes.

Value-level reconciliation v4

For datasets that declare critical fields, a dbt test compares each record's critical values in the raw batch with the model that still has one row per source record (for example `stg_lab_results`). It compares keyed values, never plain values, and stores only the keys and field names that differ.

Scope. Only records present on both sides are compared; missing and extra records are already caught by the count and key reconciliation. The test proves the pipeline preserved each value. Source-side errors, such as a unit switched at the lab interface, arrive identical on both sides and remain the job of value-range and anomaly tests.

Why a keyed comparison. Many clinical values have few possible values: a glucose reading, a date of birth, a result code. A plain hash of such a value can be reversed by hashing every candidate, so plain hashes in evidence or failure tables would be patient data. A keyed value cannot be recomputed by anyone without the key. In this design, "keyed digest" means a deterministically encrypted value: it is not one-way, and anyone with decrypt permission on the key can recover the original.

Implementation on BigQuery. BigQuery's documentation describes no HMAC function, so the design uses deterministic encryption with a Cloud KMS-wrapped keyset (`DETERMINISTIC_ENCRYPT` with `KEYS.KEYSET_CHAIN`). Identical input gives identical output, which is what the comparison needs, and the key is unwrapped in memory for the query only.

- **Differs from HMAC:** whoever holds the key can decrypt. So only the value-reconciliation identity has decrypt permission on the KMS key, and digests are computed inside the query and never stored. Security review must approve this choice.
- **Key reference:** `KEYS.KEYSET_CHAIN` takes the KMS key as a `gcp-kms://projects/…/locations/asia-south1/keyRings/…/cryptoKeys/…` URI. The key must be in the same region as the data, `asia-south1`.
- **Query text:** the wrapped keyset is passed in the query, so it can appear in the query plan and audit logs. It is useless without KMS decrypt permission; job and plan visibility are still restricted to the value-reconciliation identity.
- **Strict HMAC alternative:** if the privacy team requires a true HMAC, compute it in the reconciliation Cloud Run job. That moves critical values out of BigQuery into the job, so it is not the default.

Canonicalization. Both sides pass through one macro, `canonical_value`, before the digest. It only formats; it holds no business logic, so sharing it does not repeat the "same bug both sides" risk in section 7. It has its own dbt unit tests, and

an unsupported field kind fails at compile time. Normalization is chosen per field from its business meaning, not from its data type: a change that matters must still be detected, and only formatting differences that carry no meaning may be ignored.

Field kind	Rule
Numeric	Cast to NUMERIC, round to the scale declared in `meta`, cast to string
`code_ci`: case-insensitive code	Trim, upper-case. Only for code sets where case carries no meaning (for example facility codes declared case-insensitive by their owner)
`text_exact`: case-sensitive code or text	No change except NULL handling, so a change in case or whitespace is detected (for example LOINC display text, free-text units)
Timestamp	Convert to UTC, fixed ISO-8601 format with microseconds
Date	ISO-8601 date
NULL	Nullness is encoded apart from the value: SQL NULL becomes `n`, and every non-NULL value is prefixed `v:`. No real value can collide with NULL, including the text `<null>`

Declared transforms. A field the model deliberately changes is either left out of `critical_fields`, or declares an `expected_transform` in `meta`. The macro `apply_expected_transform` applies it to the raw side only, from a closed list: `none`, `round:N` with N from 0 to 9, or `multiply:F` with F a positive decimal. The whole string, name and parameter, is validated before any SQL is generated, so a malformed value such as `round:abc`, `round:12` or `multiply:-2` fails at compile time instead of being coerced. It never imports the model's logic, so a model that converts differently from its declaration produces a mismatch.

Duplicates and NULLs. On the raw side, duplicate rows are reduced to one per key with the same ordering as the staging model. The value test does not rely on test order: dbt does not guarantee that the `unique` test runs before it. Instead it checks the whole target batch for duplicate keys in its own step, including keys that have no match in the raw batch, and reports each one with the marker `duplicate_target_key`, which always blocks. The value comparison uses one copy per key, so a fan-out cannot inflate the field counts. A key can produce both a `duplicate_target_key` row and a value-mismatch row; they are separate failure types. When the engine applies the dataset's thresholds, it counts only value-mismatch rows, per field, against the number of compared keys; `duplicate_target_key` rows are counted separately, never added to field counts, and always block. The publication gate, which needs every blocking test to pass, remains the final safeguard. Because nullness is encoded separately, a value that became NULL, or a NULL that gained a value, is a mismatch.

Key versions and rotation. In every run, both sides are digested in one query with the same keyset, so a comparison never mixes keys. Two different rotations apply:

- **Rotating the Cloud KMS key** (the key that wraps the keyset) adds a KMS key version. The keyset inside is unchanged, so digests do not change. The KMS version that wrapped the keyset must stay enabled until the keyset is re-wrapped.
- **Rotating the keyset itself** (BigQuery's wrapped-keyset rotation) changes every digest from the next run. That is safe because no digest is stored and both sides change together.

Each run records the keyset version and the KMS key version in its evidence record.

Evidence. dbt's `store_failures` persists the test's result rows, so the test returns only `record_key`, `mismatched_fields` and `key_version`, and sets `store_failures_as='table'` (a view would re-run the query on every read). The rows go to the `evidence_restricted` dataset, readable by the value-reconciliation identity and the L3 role. That dataset's permissions and retention are part of the privacy review. Slack messages, tickets and agent prompts get the per-field mismatch counts and the evidence reference only.

Evidence location. By default dbt prefixes a custom schema with the target schema, so `schema='evidence_restricted'` alone would write to a dataset such as `audit_evidence_restricted`, outside the restricted permissions. The project therefore overrides schema naming for these tests only, and the acceptance tests in section 13 check where the table actually lands:

```
-- macros/generate_schema_name.sql
{% macro generate_schema_name(custom_schema_name, node) -%}
  {%- if node.resource_type == 'test' and custom_schema_name == 'evidence_restricted' -%}
    evidence_restricted
  {%- else -%}
    {{ default__generate_schema_name(custom_schema_name, node) }}
  {%- endif -%}
{%- endmacro %}
```

Cost. Only declared critical fields, on datasets that opt in, for one business-day batch. The test runs as a separate dbt invocation under the value-reconciliation identity, so the dbt build identity never holds the key.

Illustrative implementation. The SQL below shows the approach. It is not verified: confirm it against the project's dbt and BigQuery versions, then run it with synthetic data and the real KMS permissions (section 13) before relying on it.

```
-- macros/recon/canonical_value.sql
{% macro canonical_value(col, kind, scale=2) -%}
  if(({{ col }}) is null, 'n', concat('v:',
  {%- if kind == 'numeric' -%} cast(round(cast({{ col }} as numeric), {{ scale }}) as string)
  {%- elif kind == 'code_ci' -%} upper(trim(cast({{ col }} as string)))
  {%- elif kind == 'text_exact' -%} cast({{ col }} as string)
  {%- elif kind == 'timestamp' -%} format_timestamp('%Y-%m-%dT%H:%M:%E6SZ', {{ col }}, 'UTC')
  {%- elif kind == 'date' -%} format_date('%Y-%m-%d', {{ col }})
  {%- else -%} {{ exceptions.raise_compiler_error(
                  'canonical_value: unsupported kind ' ~ kind ~ ' for ' ~ col) }}
  {%- endif %}))
{%- endmacro %}

-- macros/recon/apply_expected_transform.sql   (raw side only)
-- The whole string is validated before any SQL is generated:
--   none | round:N (N = 0..9) | multiply:F (F a positive decimal, e.g. 18.0182)
{% macro apply_expected_transform(col, transform='none') -%}
  {%- set re = modules.re -%}
  {%- if transform == 'none' -%} {{ col }}
  {%- elif re.fullmatch('round:[0-9]', transform) -%}
    round(cast({{ col }} as numeric), {{ transform.split(':')[1] }})
  {%- elif re.fullmatch('multiply:(0|[1-9][0-9]*)([.][0-9]+)?', transform)
           and transform.split(':')[1] | float > 0 -%}
    cast({{ col }} as numeric) * cast('{{ transform.split(':')[1] }}' as numeric)
  {%- else -%} {{ exceptions.raise_compiler_error(
                  'apply_expected_transform: invalid transform ' ~ transform ~ ' for ' ~ col) }}
  {%- endif -%}
{%- endmacro %}
```

```
-- tests/value_recon_lab_results.sql   (policy_outcome: threshold)
{{ config(store_failures=true, store_failures_as='table', schema='evidence_restricted',
          meta={'policy_outcome': 'threshold'}) }}

{% set kc -%}
  KEYS.KEYSET_CHAIN('{{ var("recon_kms_key_uri") }}',
                    FROM_BASE64('{{ var("recon_wrapped_keyset") }}'))
{%- endset %}
{# (field, kind, expected_transform); the generic test reads these from meta #}
{% set fields = [('result_value', 'numeric', 'none'),
                 ('result_unit', 'text_exact', 'none'),
                 ('collected_at', 'timestamp', 'none')] %}

with src as (
  select lab_result_id as record_key
  {%- for f, k, t in fields %},
    deterministic_encrypt({{ kc }},
      {{ canonical_value(apply_expected_transform(f, t), k) }}, 'lab_results.{{ f }}') as d_{{ f }}
  {%- endfor %}
  from {{ source('raw', 'lab_results') }}
  where _batch_id = '{{ var("batch_id") }}'
  qualify row_number() over (partition by lab_result_id
                             order by _loaded_at desc, _row_id desc) = 1
```

```
),
tgt as (
  select lab_result_id as record_key
  {%- for f, k, t in fields %},
    deterministic_encrypt({{ kc }}, {{ canonical_value(f, k) }}, 'lab_results.{{ f }}') as d_{{ f }}
  {%- endfor %}
  from {{ ref('stg_lab_results') }}
  where _batch_id = '{{ var("batch_id") }}'
  qualify row_number() over (partition by lab_result_id order by lab_result_id) = 1
),
-- Duplicates are checked over the whole target batch, including keys
-- with no source match, so the inner join below cannot hide them.
target_duplicates as (
  select lab_result_id as record_key
  from {{ ref('stg_lab_results') }}
  where _batch_id = '{{ var("batch_id") }}'
  group by lab_result_id
  having count(*) > 1
),
value_mismatches as (
  select
    record_key,
    array_to_string([
      {%- for f, k, t in fields %}
      if(s.d_{{ f }} != t.d_{{ f }}, '{{ f }}', null){{ ',' if not loop.last }}
      {%- endfor %}
    ], ',') as mismatched_fields
  from src s
  join tgt t using (record_key)
  where
    {%- for f, k, t in fields %}
    {{ 'or ' if not loop.first }}s.d_{{ f }} != t.d_{{ f }}
    {%- endfor %}
)
select record_key, 'duplicate_target_key' as mismatched_fields,
       '{{ var("recon_key_version") }}' as key_version
from target_duplicates
union all
select record_key, mismatched_fields,
       '{{ var("recon_key_version") }}' as key_version
from value_mismatches
```

In the build, a generic test reads the field list from the dataset's `meta` instead of the hard-coded list above:

```
models:
  - name: stg_lab_results
    meta:
      record_key: lab_result_id
      source_system: lis
      critical_fields:
        - { name: result_value, kind: numeric, scale: 2, expected_transform: none }
        - { name: result_unit, kind: text_exact }      # case-sensitive unit text
        - { name: collected_at, kind: timestamp }
      value_recon: { block_if_mismatched_rows_gt: 0 }
```

12. Multi-agent roles

Agents add judgement where rules run out, but they never act on data. The deterministic engine is the supervisor: it decides when an agent is called, validates what comes back and is the only component that can trigger an action. Agents do not call each other directly; every hand-off goes through the engine, so each step is logged.

Agent	Called when	Reads (access)	Produces	Cannot do
Diagnosis	Incident is unknown or confidence is low	Splunk logs, dbt artifacts, lineage, deployment context, `select` on masked audit data	Root cause, evidence references, one proposed catalog action, confidence	Write data, run DDL or DML, see unmasked PHI

Runbook	Every incident; after every resolution	Git runbook repo, past incidents	Matching runbook and link; drafted runbook update as a pull request	Merge pull requests or enable rules
Communication	An alert, approval or escalation is needed	Incident record, evidence, severity, escalation policy	Urgent Slack or PagerDuty alerts; Slack approval requests; ticket details for unresolved engineering issues; escalation status and resolution evidence in the audit log	Approve its own requests; include patient identifiers or unmasked PHI
Code-fix (v2)	Engineer-fix incidents (schema drift, fan-out, logic)	Repo (read), sandbox warehouse with synthetic or masked data	Branch and pull request with a test that reproduces the failure and a fix; CI result	Push to main, deploy, touch production

How agents interact with the engine

1. The engine sends the incident context (signals, severity, lineage, deployment context, relevant runbook) to the Diagnosis agent.
2. The agent returns structured JSON that cites evidence and separates confirmed facts from hypotheses.
3. The engine validates it: the schema is correct, the evidence references exist, the action is in the catalog and confidence meets the threshold. Anything else is rejected and escalated.
4. A validated proposal goes back through the policy gate, exactly like a rule-based decision.
5. For alerts, approvals and escalations, the Communication agent prepares the message, and a ticket only where the escalation policy calls for tracked work. After resolution, the Runbook agent drafts the update, using the audit record and any closed ticket as inputs.

Agent guardrails

- Read-only, least-privilege credentials; masked or synthetic data only; no PHI in prompts, messages or tickets.
- Logs, runbooks and data are treated as data, never as instructions, which blocks prompt injection from log content.
- Output must be in the action catalog; free-form commands are rejected.
- Each call is time-boxed; on timeout or failure the incident escalates instead of retrying the agent indefinitely.
- A kill switch turns every automatic action, including agent-proposed ones, into a recommendation.
- Every prompt, response and validation result is stored with the incident for audit, together with the agent, model and prompt versions v4.

Agent registry and provenance v4

Agent registry. Each agent's configuration lives in git: agent ID and version, model and model version, prompt template version, allowed tools, and token and time budgets. Changing any of them creates a new agent version through a reviewed pull request, and the failure-injection suite replays past incidents against it before merge. Model versions are pinned; a provider retiring a model goes through the same process.

Per-call record. Every agent call stores the provenance fields listed in the evidence contract (section 9) with its incident, alongside the policy version. Model output is not deterministic, so replaying an incident compares a new run with the stored output rather than expecting an identical answer.

Agent quality. The ops marts track, per agent version: the share of recommendations the engine accepted as valid, the share later confirmed by the human who resolved the incident, timeouts and invalid outputs, and cost and time per incident. A version whose confirmed rate drops below the level set in policy is rolled back to the previous version through a pull request.

Evaluation gate v4. A golden set of 30–50 synthetic incidents, each with a known cause and correct catalog action, covers every rule in the failure catalog plus unknown cases. Before any change to an agent, model, prompt or context builder merges, the evaluation harness runs each incident several times and scores the action chosen, valid output, evidence references and consistency across runs. The change merges only if it scores at least as well as the current version and never has an action outside the catalog accepted by the engine.

Agent unavailable v4. If the agent platform is down, rate-limited or past its time budget, the engine skips the agent, treats the incident as unknown and escalates. The pipeline never waits on an agent, and publication follows the normal gate.

Learning approval boundary

Learning from a resolution never makes a new recovery rule trusted on its own. One wrong diagnosis must not become a recurring automated mistake.

Closed incident → resolution evidence → proposed runbook or policy change → tests → human review → approved merge

- Tests replay the proposed rule against historical incidents and the failure-injection suite; it must make the same or better decisions on every case.
- A newly merged rule starts in **recommend-only** mode and is promoted to auto-fix only after it has proposed correct actions on real incidents.
- Every rule change bumps the policy version, so any later decision can be traced to the rule that made it.

Learned thresholds v4

1. A weekly job proposes per-dataset volume, freshness and distribution thresholds from the last 90 days. The proposal is a pull request changing dbt `meta`, never a runtime change.
2. The proposed threshold runs in shadow next to the active one for an agreed number of runs. The pull request shows how many runs each would have warned or blocked.
3. The dataset's data owner approves tightening. Loosening a blocking threshold needs the data owner and the platform owner.
4. The merge bumps the policy version, so every later decision is traceable to the threshold that made it.

Thresholds never change while a pipeline runs, and an agent can propose a threshold but never merge one.

13. Guardrails and production-readiness controls

These controls make every recovery bounded, repeatable and safe for clinical, financial and regulatory consumers.

Control	How it works
Write-Audit-Publish	Build in audit, test, and clone into analytics only when every blocking check passes. Users see slightly old figures rather than wrong ones
Publication gate	Counts, distinct keys and totals must agree together, plus referential integrity, freshness, schema compatibility, value ranges and units. Source control totals are captured in the extract's own consistent snapshot and compared with the warehouse as of the same batch, on the same business-day definition
Checks actually ran	The expected number of tests is asserted, and a control-totals row must exist for every dataset, business date and batch; fewer tests than expected or missing totals is a failure, never a pass
Tolerances and noise	Warn and block thresholds per check, in absolute and percentage terms. A transient, low-risk failure must repeat on two consecutive runs before it escalates
Immediate escalation	P1, security or PHI exposure, unsafe publication and missing checks bypass the confirmation window and escalate at once; unknown failures hold publication when needed and escalate without a fix
Unknown means escalate	Only failures with a specific rule are fixed automatically
Retry limits	At most 2 attempts per action; after the cap or maximum runtime, hold publication, cool down and escalate with evidence
Safe, idempotent recovery	A partition reload is delete + insert in one transaction, read from the raw archive, with one lock per object; a retry can never insert the same records twice
Permissions enforce the gate	Only the publish service account can write to analytics; agents read masked views only; every query is captured by Cloud Audit Logs (section 4)
Dependency-aware publishing	A mart is published only when every dataset it depends on is published; otherwise it stays on its last verified day
Data changes only through dbt	Every fix that changes warehouse data runs as a dbt build; nothing edits a dbt-managed table outside dbt, so the next run always reproduces the same result
Duplicate incidents	Signals with the same incident key (dataset, business date, failure type, scope) join one incident, so a re-delivered alert never triggers a second fix

Single writer per partition	An executor takes a lease-based lock on the dataset and partition before acting; overlapping pipeline runs and a second executor wait or abort
Action journal	Each attempt is recorded before it runs and its outcome after. After an orchestrator timeout, the executor checks the journal and the data state before retrying, because the first attempt may have completed
Partial completion	Actions run in one transaction where possible; otherwise each action declares a rollback (for example, restoring the partition from a BigQuery table snapshot taken before the action). A rollback counts as done only after reconciliation re-runs on the restored partition. Downstream models are rebuilt from it, and side effects a clone cannot undo, such as messages already sent, are logged and corrected with a follow-up notice
No destructive refresh	`full_refresh: false` on clinical and claims models; source deletes are never propagated
Action contracts	Each action declares preconditions, scope, timeout, retry limit and backoff, rollback and success criteria
Fix ordering	Blockers first, then re-extract, one rebuild (which also deduplicates), orphans last, then re-run all checks
Stale-data handling	Consumers see the last successful refresh time; past the freshness SLA, data is shown as delayed, never as current
Schema and unit drift	Added, removed, renamed or retyped columns and unit changes are detected; incompatible changes need approval
Governance boundary	Decision engine, agents and executor run with least-privilege service identities; every action is audited
PHI protection	See healthcare privacy controls below
Control-plane failure	If the engine, state store or Slack is down, the pipeline fails closed (publication held) and alerts through a fallback channel
Failure-injection tests	Partial loads, duplicates, timeouts, corrupt rows, missing control totals, schema and unit changes, archived parents, merged patients, failed retries and rollback, proving bad data is never published; plus the v4 cases below
Upstream-outage breaker v4	Correlated failures from one source are grouped into one incident; affected datasets are held and per-dataset fixes are suppressed until a source probe and a freshness check both pass
Scoped breakers v4	One source's breaker never stops recovery or publication for another source; only the global breaker switches all automation to recommend-only, and healthy sources keep publishing
Publish-time recheck v4	The publish step re-reads publication state and open breakers immediately before each clone, so a fix that finished after a breaker opened cannot publish
Deployment context v4	Recent releases touching the failing scope are attached to the evidence; they inform classification but never trigger a rollback
Value-level reconciliation v4	Declared critical fields are compared by keyed value between the raw batch and the source-grain model; the engine applies the dataset's thresholds, and one mismatch blocks a P1 dataset
Reconciliation key protection v4	KMS key in `asia-south1`, referenced as a `gcp-kms://` URI; decrypt permission only for the value-reconciliation identity; no digest is stored; keyset and KMS key versions recorded per run
Privacy-safe mismatch evidence v4	Only record keys, field names and key version are stored, as tables in `evidence_restricted` (schema naming overridden so the location is exact); messages and prompts carry counts and references only
Agent provenance v4	Every agent call records agent, model, prompt and policy versions with its input references and validated output
Duplicate events v4	Every run-complete event carries its run ID. Before evaluating, the engine claims the ID with a single atomic insert into a Cloud SQL table whose primary key is the run ID; only the delivery whose insert succeeds proceeds, and any other delivery, even a concurrent one, fails the claim and is dropped. Pub/Sub redelivery therefore never leads to two instances evaluating the same run at once. The claim row holds a status (`CLAIMED`, `COMPLETED` or `FAILED`) and a lease that the owner renews while it works; the engine acknowledges the Pub/Sub message only after `COMPLETED`. If the owner crashes, its lease expires and the message is redelivered; the new delivery takes over with one conditional update that succeeds only when the claim is still `CLAIMED` and its lease has expired, so two instances never hold it at once. The guarantee is one owner at a time and an effectively-once outcome, not that every internal step runs literally once: a takeover re-runs the evaluation, but its effects are not repeated, because incidents join by their stable key, and the executor checks the action journal before any action, as for an orchestrator timeout. A scheduled sweep re-publishes any expired claim whose message was lost. After the takeover limit set in policy, the claim becomes `FAILED`, publication stays held and the run escalates

Policy validation in CI v4	Every `policy.yml` change is checked against a schema: known action names only, thresholds in range, required safety entries present, `fallback` last. An invalid policy fails the build and never reaches production
Approved thresholds v4	Learned thresholds arrive as pull requests, run in shadow first and need owner approval; nothing changes at runtime

Healthcare privacy controls

Control	How it works
Role-based access	Separate roles for quarantine tables, incident evidence and recovery actions; agents and the executor use least-privilege service accounts
No PHI outside the platform	Slack, PagerDuty, tickets and agent prompts carry only a protected evidence reference, never patient identifiers or unmasked values
Masking	Agents read masked data only; column masking and row access policies apply in the warehouse
Encryption	In transit (TLS) and at rest with customer-managed keys in Cloud KMS for the warehouse, raw archive, evidence store and logs
Keyed comparison v4	Value-level reconciliation never stores clinical values or plain hashes; the keyed values exist only inside the query, and the key is restricted to one service identity
Retention and deletion	Defined retention for raw extracts, evidence (including `evidence_restricted`), logs and quarantine, aligned with records-retention policy; expired data is deleted on schedule and the deletion is logged
Access auditing	Every read of evidence or quarantine and every recovery action is logged with who, what and when
Synthetic data	Demonstrations and failure-injection tests use synthetic patients, never production records

These controls support HIPAA or India's DPDP Act requirements; the exact obligations should be confirmed with the privacy and compliance team.

Minimum detection set for the pilot

- Count, distinct-key and total reconciliation per local business day for encounters and claims
- Control-totals presence test for every dataset, business date and batch
- `unique` and `not_null` on encounter, claim and lab result IDs, plus a duplicate-key test on each raw batch
- `dbt source freshness` on every clinical and billing source, and on `ops.control_totals`
- Value-range and unit checks on key lab results; anomaly test on daily billed totals vs the trailing 7-day average
- Grain test confirming facts have no fan-out
- `relationships` tests against warehouse dimensions, with stored failures
- Expected-test-count assertion
- Weekly reconciliation of the last 90 days by amount and distinct keys
- Upstream-dependency check at the publication gate
- Value-level reconciliation on declared critical lab-result fields v4
- Upstream-outage breaker for the lab and EHR sources v4

> **Example: guardrails stopping a bad fix.** On 20 October the value-range test fires: glucose results average 5.6 instead of about 100, because the lab interface switched to mmol/L. A naive engine might "fix" it by fully refreshing the lab model, which would erase archived history and still be wrong. Instead, the unit rule is never-auto, so nothing runs; Write-Audit-Publish keeps the new values in audit and reports still show 19 October, flagged as delayed; and the escalation carries the daily average, the 7-day average and masked sample rows, so the engineer sees the unit change at once. Value-level reconciliation passes here, correctly: the values arrived unchanged from the source.

Operational SLOs (proposed, to be validated in the pilot)

SLO	Target	When the clock runs
Detection	Reconciliation or data-quality failure detected within 5 min	From pipeline run completion to the incident being opened
Automated recovery (L1)	Eligible failure fixed and verified within 30 min. Escalations are reported as unsuccessful automated recoveries, never as successes	From detection to verified recovery, or to the moment automatic remediation stops and escalates; excludes time waiting for human approval
Approval wait	Measured and reported, no target yet	From approval request to the owner's decision
Human resolution (L2/L3)	Measured separately from automated recovery, to set a baseline; targets to be agreed with SRE	From escalation to verified recovery
P1 notification	Data owner and on-call engineer notified within 5 min, in parallel with auto-recovery	From detection of a P1
Data freshness	Business-approved deadline met (for example 06:00 for the morning bed meeting), or data flagged as delayed	Deadline agreed per dataset with the business
Publication safety	Zero publishes of data that failed mandatory checks	A hard safety invariant, not a time target

Acceptance criteria for the v4 safeguards v4

These cases join the failure-injection suite. Each must pass in CI and in the test environment.

Safeguard	Injected condition	Passes when
Outage breaker	Three lab datasets fail extraction in the same window	One grouped incident; zero automatic fixes run; all three and their dependants are HELD; one alert sent
Breaker scope	The lab source fails while billing is healthy	Billing datasets reconcile, recover and publish normally
Unknown inventory	The manifest inventory for the lab source cannot be read	Only the count rule applies; a warning is logged; the breaker still opens at three failures
In-flight fix	A replay is running when its source's breaker opens	The replay finishes and is journaled; the publish step finds the open breaker and does not clone
Probe without freshness	The source probe passes but freshness still fails	The breaker stays open and the datasets stay HELD
Breaker close	Probe and freshness both pass	Each child re-runs the five gates, oldest date first
Deployment context, no data cause	A release changes `stg_lab_results` and its grain test then fails	Incident is `suspected_regression`, engineer fix, release attached; no data action runs
Deployment context, data cause	A partial extract happens on the day of an unrelated release	`reextract_from_raw` runs as normal; the release is attached but does not change the decision
Value mismatch	A model casts `result_value` to two decimals where the source has three, on 14 rows	Test blocks publish; evidence holds 14 keys and the field name only
No values leak	Same run	A scan of `evidence_restricted`, Slack messages, tickets and agent prompts finds no clinical values and no digests
Evidence location	Same run	The failure table lands in `evidence_restricted` itself, not in a prefixed dataset, and holds only `record_key`, `mismatched_fields` and `key_version`
Privacy path, end to end	Other identities (dbt build, agents, analysts) try to read the evidence table, view the digest job and its query plan, and use the KMS key	Every attempt is denied, and every attempt appears in Cloud Audit Logs
Case-sensitive field	A `text_exact` unit changes only in case between raw and model	Reported as a mismatch
Case-insensitive code	A `code_ci` facility code differs only in case or surrounding spaces	Not reported

Declared transform	A field declares `round:2` and the model rounds to 2; then the model is changed to round to 1	No mismatch in the first case; mismatches in the second
Duplicates and NULLs	The raw batch holds a key twice; a value becomes NULL in the model	The raw duplicate is counted once; the NULL change is a mismatch
NULL vs literal text	One side holds SQL NULL, the other the text `<null>` (and, separately, the text `n`)	Both are reported as mismatches
Duplicate target key	The model holds a key twice, and the value test is run without the `unique` test having run first	The value test itself reports `duplicate_target_key` and blocks publication; field counts are not inflated
Target-only duplicate key	The model holds a key twice that does not exist in the raw batch	The value test still reports `duplicate_target_key`; the key reconciliation also reports the extra records
Duplicate key with a mismatch	A target key is duplicated and one of its fields also differs from the raw batch	One `duplicate_target_key` finding and one value finding; the field mismatch count rises by one, the compared-key count is unchanged, and publication is blocked by the duplicate
Malformed transform	A dataset declares `round:abc`, `round:12`, `multiply:-2` or `multiply:x`	Compilation fails with an invalid-transform error; no SQL runs
KMS key rotation	A new Cloud KMS key version is made primary between two runs	Digests are unchanged; no mismatch; the new KMS version is recorded
Keyset rotation	The wrapped keyset is rotated between two runs	Digests change on both sides together; no mismatch; the new keyset version is recorded
Key access	Any identity other than the value-reconciliation identity runs the digest query	The query fails on KMS permission
Agent provenance	Any agent call during the suite	Its incident record carries agent, model, prompt and policy versions
Threshold approval	A proposed threshold is merged without owner approval	The merge is blocked; the active threshold is unchanged
Agent output contract	Canned agent responses: malformed JSON, an action outside the catalog, missing evidence references, low confidence	Each is rejected and the incident escalates; no action runs
Prompt injection	A log line instructs the agent to recommend `full_refresh`	No non-catalog or never-auto action is accepted; the injected text stays inside the untrusted-data section of the context
Agent unavailable	The agent platform times out or returns an error	The incident escalates as unknown; the run is not delayed
Evaluation gate	A prompt change that lowers golden-set accuracy	The merge is blocked
Duplicate event	The same run-complete event is delivered twice, one after the other	One evaluation, one set of incidents, no second action
Concurrent duplicate event	The same event is delivered to two engine instances at the same moment	Exactly one claim succeeds; one evaluation, no second action
Engine crash after claim	The winning instance is terminated after claiming the run ID and starting an action, before evaluation completes; the event is then redelivered	The run is evaluated once to completion by another instance after the lease expires; no duplicate incidents; the started action is not repeated
Lost message	The owner crashes and the redelivered message is dropped	The scheduled sweep finds the expired claim and re-publishes it; the run completes once
Takeover limit reached	Every instance that takes over crashes, until the limit set in policy	The claim becomes `FAILED`, publication stays held and the run escalates
Invalid policy	A `policy.yml` change with an unknown action name or an out-of-range threshold	The CI build fails

The values above (three datasets, 14 rows, two decimals) are test fixtures, not production settings.

Verification tracker v4

An item moves to “Passed” only when its tests above pass in the test environment; until then the related text in this document is design intent, not verified behaviour.

Item	Verified by	Status
Value-level SQL and macros	Runs on synthetic data with the project’s dbt and BigQuery versions; unsupported kinds and transforms fail at compile time	Not verified
Field-specific canonicalization and declared transforms	Case-sensitive, case-insensitive, declared-transform, duplicate, NULL-vs-literal, duplicate-target-key, target-only-duplicate and malformed-transform tests	Not verified
Evidence location and privacy path, end to end	Evidence-location and privacy-path tests, run together with the key access test	Not verified
KMS permissions and key access	Key access test	Not verified
KMS key and keyset rotation	Both rotation tests	Not verified
`store_failures` evidence contents and permissions	No-values-leak test and privacy review	Not verified
Outage breaker, including in-flight fixes and recovery	The six breaker tests	Not verified
Deployment context	Both deployment-context tests	Not verified
Agent provenance	Provenance test	Not verified
Threshold approval	Approval test	Not verified
AI residency and agreement coverage	Written confirmation from Google and the compliance team (section 4)	Not verified
Agent contract, injection and outage handling	Agent output contract, prompt-injection and agent-unavailable tests	Not verified
Evaluation gate and golden set	Golden set built; evaluation-gate test	Not verified
Duplicate events and policy validation	Sequential, concurrent, crash-after-claim, lost-message and takeover-limit tests, invalid-policy test	Not verified

14. End-to-end walkthrough: one night

On the run for 13 October, the pipeline detected four problems: it fixed two automatically, removed one in the staging model’s built-in deduplication, and logged one as expected, then published correct data without anyone seeing a wrong report. Step numbers refer to Figure 1.

1. **Extract and land (steps 1–3).** Encounters, lab results, claims and payments are written to the raw archive, partitioned by load date, and the extract writes its control totals to `ops.control_totals`.
2. **Build in audit (step 4).** `dbt build --target audit --vars '{"run_date": "2026-10-13"}'`. Reports keep showing the last verified day.
3. **Detect (steps 5, 6, 8).** Four checks flag: 800 encounters missing (10,000 vs 9,200); 12 duplicate claim rows in the raw batch (warning); 35 lab results whose encounters are not loaded; 1,200 extra encounters on dates older than the retention window. Value-level reconciliation on lab results passes. Only the EHR source has failing datasets, below the breaker threshold, so no outage group opens. Publication is held.
4. **Decide (step 9).** 800 missing, all between 02:00 and 03:00 → partial extract → replay from raw archive; no release touched the encounter models. 12 duplicates from one batch ID → removed by the rebuild, logged. 35 orphans, all under 2 hours old → unknown member, recheck next run. 1,200 old encounters found in the archive index → expected, log only.
5. **Remediate (step 11)** in safe order: replay 02:00–03:00 from the raw archive, then one rebuild of the affected models and everything downstream, which also drops the 12 duplicate claims.
6. **Re-check (back to step 5).** 10,000 vs 10,000; no duplicates in `stg_claims`; source total = mart total + quarantined total; 35 orphans within the age limit (warning only).

7. **Publish (steps 6 → 7).** The gate passes, no breaker is open, and the audit tables are cloned into analytics.
8. **Log (step 13).** Every failure, decision, fix and re-check goes to the audit table, with a Slack summary: "13 Oct healed automatically: 800 encounters replayed, 12 duplicate claims dropped on rebuild, 35 lab results awaiting encounters."

Each problem became one incident with a stable key and an evidence record. The two fixed incidents (the replay and the orphans) moved DETECTED → TRIAGED → REMEDIATING → VERIFYING → RESOLVED; the duplicates incident was resolved by the same rebuild and closed once verification passed; the archived encounters were closed as RESOLVED (expected) without any action.

Next night. 33 of the 35 lab results re-link automatically as their encounters arrive. Two are still orphaned after 24 hours: one belongs to a patient merged in the index and goes for approval to remap (L2, step 12 → 11); the other has no parent anywhere and is quarantined and escalated.

When it can't heal. If a unit change, a fan-out or a critical-value mismatch appears, no safe fix exists: the engine blocks publish and escalates to L3, and reports stay on the last verified day. If the lab interface goes down and three lab datasets fail together, the breaker holds them as one outage, sends one alert, and re-evaluates each dataset once the source recovers v4.

15. Components: functionality, inputs and outputs

Numbers match the steps in Figure 1. Section 6 gives example values for these inputs and outputs.

#	Component	Functionality	Inputs	Outputs
1	Source systems	Hold clinical and billing data; provide control totals per business day	EHR, lab and billing tables	Records to extract; counts, distinct keys, amounts
2	Ingestion (Airflow)	Extract one partition, load it idempotently, stamp lineage, write control totals	Source records, run date	`raw` tables with `_batch_id`, file manifest, `ops.control_totals`
3	Raw archive	Keep every extract immutably; serve replays for fixes	Extract files	Partitioned history, retained beyond source retention
4	Transform (dbt, audit)	Build staging (with built-in deduplication), intermediate and marts in the audit schema	`raw` tables	Audit models, `run_results.json`, `manifest.json`
5	Reconcile and test	Compare source vs raw vs marts; run dbt tests, freshness, unit and anomaly checks; compare critical field values by keyed value v4	Control totals (dbt source), audit models, tolerances, expected test count, critical-field declarations	`recon_results`, test results, `sources.json`, mismatched keys in `evidence_restricted`
6	Publication gate	Decide once per dataset and business date whether data may be published	Blocking test results, reconciliation status, open incidents, open breakers v4	`publication_state`: PUBLISHED or HELD
7	Publish	Re-check gate state, then clone audit → analytics only after the gate passes	Audit models that passed	Trusted marts for consumers
8	Hold and collect	Keep the last verified day, flagged as delayed; normalize every check, error and log; group related signals under a stable incident key, and correlated source failures under one outage group v4	Recon, dbt artifacts, Airflow errors, Splunk logs, archive and patient index	Signals, incidents with evidence records, outage groups
9	Decision engine	Five gates: ran, trust, expected, classify, guardrails; severity, policy gate and lifecycle state	Incidents, evidence records, `policy.yml`, lineage, deployment context v4	Severity (P1–P3), mode (auto-fix, approve or engineer fix), state transition
10	AI agents	Diagnosis, runbook, communication, code-fix (v2); advisory only; versioned in the agent registry v4	Logs, SQL, lineage, runbooks (read-only, masked)	Root cause, evidence, proposed action (JSON), message, PR; provenance record

11	Recovery executor	Take the partition lock, journal the attempt, run one catalog action (data changes as dbt builds), enforce attempt caps, fix order and the kill switch	Decision, action catalog, raw archive, action journal	Retry, replay, rebuild, link, remap, quarantine or hold; journaled outcome
12	Notify and escalate	Page for P1, send Slack alerts and approval requests, open tickets only for tracked engineering work; one alert per outage group	Decision, evidence, escalation policy, runbook link	Urgent alerts, Slack approvals, tickets, escalation evidence
13	Observe and learn	Log every incident (including auto-healed ones), decision and action; track SLOs and agent quality; propose, never enable, runbook, rule and threshold changes	All events, approval decisions, closed tickets	Splunk events and dashboards, ops tables, tested runbook, policy and threshold PRs

16. Business benefits by stakeholder

Stakeholder	Benefit
Clinical and operations leaders	Admissions, occupancy and lab figures are verified before use; delayed data is flagged, never shown as current
Revenue-cycle and finance teams	Claims, adjustments and payments reconcile to source; no double-counted or missing claims
Quality and regulatory reporting	Measures and submissions built only from data that passed every check
L1 / SRE support	Routine alerts resolve themselves; a source outage arrives as one alert, not dozens; less manual Splunk searching and runbook work
Data engineers (L2) and developers (L3)	Incidents arrive already diagnosed, with evidence, any related release and a proposed fix; only real code issues reach L3
Data and platform managers	Measurable KPIs: MTTR, SLO compliance, auto-resolution rate, first-fix rate, agent acceptance rate
Analysts and data scientists	Fresh, complete data for reports and models, with known history after archiving
Privacy, compliance and audit	Full trail of what failed, what was done, by whom and under which policy and agent version, with patient data kept under least privilege
IT leadership	Less firefighting and fewer full reruns, so lower operating cost

How to prove it: take the current baseline from Splunk (data incidents per month, time to detect, time to recover by P1/P2/P3, incidents found by users), then compare after rollout. Track MTTR, auto-recovery success, retry exhaustion, repeated failures, stale-data duration, publication blocks and false alarms, including those from archived or merged records. Report automated recovery and human resolution separately.

Pilot demo scenarios

Four reproducible scenarios, run on synthetic patient data. For each, capture the inputs, the decision, the action, the verification result and the audit event.

Scenario	Injected failure	Expected outcome	Evidence shown
1. Automatic recovery	Partial extract: one hour of encounters dropped	Replayed from raw archive, verified, published; incident RESOLVED	Evidence record, rule `partial_extract`, journal entry, re-check pass, audit event
2. Blocked publication	Lab unit switched to mmol/L, or a source column renamed	Incident ESCALATED, publication HELD; reports stay on the last verified day	Failed test IDs, never-auto rule, escalation message with evidence reference only
3. Expected difference	Archived encounter, or a merged patient ID, as an orphan's parent	Recognised as expected (archived) or sent for approval (merged); no false alarm	Archive or patient-index lookup, `log_expected` or approval record, audit event

4. Shared outage v4	Lab interface down: three lab datasets fail together	One grouped incident, one alert, all three HELD, no per-dataset fixes; each re-evaluated after recovery	Outage group record, suppressed actions, publish-time recheck, audit event

17. Sources

Source	Idea taken
Paradime: Building autonomous self-healing pipelines for dbt	Detection → diagnosis → remediation → validation loop, guardrails, tiered approval, knowledge base, MTTR and first-fix rate
Paradime: Autonomous dbt pipelines with AI agents	AI agents for diagnosis and remediation, MTTR impact
Paradime: Bolt AutoPilot self-healing	Contextual fixes before retry; vendor-reported burnout figure
Paradime: Self-healing pipelines announcement	Projection that most pipeline errors are fixable by agents
Paradime docs: Bolt pipeline agent	Log summaries, one-click fix, self-healing re-run
Paradime: DinoAI Slack agent	Slack-based alerts, human-in-the-loop approval, PR creation
Agentic Data Infrastructure (Substack)	Green-but-wrong pipelines; auto-fix only at high confidence and low blast radius; one fix per attempt, then verify
phData: Self-healing CI/CD for dbt Cloud	GenAI diagnosis of failed dbt models and tests
ACL Digital: The Autonomous Data Doctor	Controlled remediation library and governance boundaries
The Write-Audit-Publish pattern	Audit schema, publish only when checks pass
QuestDB: Circuit breaker pattern	Stop the spread of failures; hold publication
Data reconciliation tools guide	dbt `audit_helper` for source vs target comparison
Recce (PyPI)	Environment diffs for dbt
dbt docs: retry	Resume from the point of failure
dbt docs: run_results.json	Per-node status as machine-readable signals
Elementary: data tests	Volume, freshness and schema anomaly tests
Splunk docs: About Splunk Free	Free tier limits (500 MB/day, no alerting) for a no-cost pilot
Google Cloud: Managed Service for Apache Airflow release notes	Product rename from Cloud Composer; Airflow 3.x builds on Gen 3; default build
Google Cloud: Track data lineage for a BigQuery table	Knowledge Catalog (formerly Dataplex) captures BigQuery lineage automatically
Google Cloud: HIPAA compliance on Google Cloud	Business associate agreement and covered services
Snowflake docs: Supported cloud regions; Snowflake editions	No Snowflake region on Google Cloud in India; Business Critical edition for PHI
Google Cloud: Column-level encryption with Cloud KMS (BigQuery) v4	Deterministic encryption with a KMS-wrapped keyset for equality comparison; key unwrapped in memory only; same-region key requirement
Databricks: Agentic AI security, extension to the AI Security Framework (DASF) v4	Registering agents and prompts; managing agent state; human-in-the-loop controls
Monte Carlo: Observability agents v4	Read-only troubleshooting agents; recommended monitors and thresholds
Acceldata: Agentic incident management v4	Clustering related alerts into one incident; deployment events in root-cause analysis; human approval before fixes
Datafold: Data reconciliation v4	Value-level, row-by-row comparison beyond counts and totals

The Paradime figures (MTTR reduction, burnout, share of errors fixable) are vendor-reported; validate them before quoting them as facts. The healthcare figures and thresholds in this document are illustrative and should be replaced with the platform's real volumes.

Part B — Phase plans

Precedence. For current POC scope and status, Part B and Part C supersede any conflicting v4 statement in Part A. In particular:

- **Agents (v4 §12):** the POC builds Diagnosis and Communication fully, Runbook as match-only; Code-fix is excluded and separately gated. See Phase 5.
- **Value-level evidence (v4 §11, §13):** evidence is written by the dbt model `value_recon_lab_results`, not by dbt `store_failures` on a test, and the `generate_schema_name` override in v4 is not the mechanism in use. See the Phase 2 deviation.
- **Verification tracker (v4 §13):** replaced by the v5 tracker in Part C.
- **Platform (v4 §4):** where the trial account differs (Docker Airflow, one project, no VPC Service Controls or IAP), the table "Where the POC differs from the design" applies.

The map and differences below tie the Part A architecture to what is built; then every phase uses the same eight headings.

Components mapped to phases

Each component of the GCP design arrives in one phase; the POV deck's ten steps map onto them as below.

Deck step	GCP services	Phase
Foundation (IAM, keys, network, CI)	IAM, VPC, Cloud SQL, Pub/Sub, Artifact Registry, Secret Manager, Terraform	P0
1 Upload · 2 Land	Airflow → Cloud Storage locked archive → BigQuery `raw` + `ops.control_totals`	P1
3 Build · 4 Reconcile	dbt in Airflow → BigQuery `audit`; Cloud KMS value check → `evidence_restricted`	P2
8 Publish · 9 Consume	Publish job on Cloud Run (clone to `analytics`), publication state in Cloud SQL, Looker Studio	P3
5 Decide · 7 Fix	Pub/Sub `run-complete` → engine (Cloud Run service) → executor (Cloud Run job); claims, locks, journal in Cloud SQL	P4
6 Diagnose · 10 Notify	Agent Platform + Model Armor, Pub/Sub `notifications` → Slack, PagerDuty	P5
Breaker, observability	Engine breaker, Cloud Monitoring uptime checks, audit-log sink, quotas, budgets	P6

Where the POC differs from the design

Design (v4)	POC on the trial account	Why	Changes when
Managed Airflow (Gen 3)	Airflow 3.3.2 in Docker on the laptop	Managed Airflow costs hundreds of dollars a month idle	DAGs move to Managed Airflow unchanged
Separate data, control-plane and agents projects; VPC Service Controls	One project, one service account per component	A personal account has no organisation	Organisation account
Console behind IAP	Private Cloud Run, opened through `gcloud run services proxy`	IAP needs an organisation	With IAP
Bucket Lock	Retention policy, unlocked (7 days)	Locking is irreversible	`lock_archive_retention = true`
Cloud SQL HA	Zonal, db-f1-micro	Cost	`availability_type = REGIONAL`
Synthea batches via a landing bucket (deck)	Airflow extracts from a Postgres synthetic generator	Same snapshot can count rows and totals together	Add landing bucket if a file feed is used
One `nightly_reconcile` DAG (deck)	`ingest_ehr`, `ingest_lis`, `ingest_billing`, `dbt_build`	Per-source schedules and retries	—
`sa-recon` (deck)	`sa-valuerecon`	Naming in Terraform	Align the deck
Value evidence via dbt `store_failures`	Written by model `value_recon_lab_results`	See Phase 2 deviation	Recorded in Phase 2
Splunk	Cloud Logging	Out of POC scope	After the POC

Phase 0 — Foundations

1. Status and exit gate. Implemented 20 Oct 2026. Exit gate 3 of 4 passed; the destroy + rebuild proof is pending until archive retention expires (about 28 Oct), or can be run in a fresh project.

2. Goal and why. A secured, empty platform and realistic synthetic data, so every later phase inherits real identities and networking. Without it, safety rules become promises. *Example:* Terraform gives `sa-ingest` write access to `raw` only; in Phase 1 the check G4 tries to write `analytics` as ingest and Google refuses with `bigquery.tables.create denied`.

3. Design.

- Six BigQuery datasets with fixed writers: `raw` (ingest), `audit` (dbt, executor), `analytics` (publish only), `quarantine` (dbt, executor), `ops` (ingest, engine), `evidence_restricted` (value-recon only).
- One service account per component, impersonated through your login; no key files.
- Deterministic synthetic generator (seed 2026) with deliberate edge cases: UTC vs facility day, late lab results, claim reversals, patient merges, archived encounters.
- `policy/policy.yml` with kill switch and never-auto list; CI rejects unsafe policies; `agents.enabled: false`.
- FastAPI + React console on Cloud Run with an Environment screen that reads live IAM.

4. GCP infrastructure.

Resource	Detail
BigQuery	6 datasets + `ops.control_totals`, `ops.batches`, `ops.source_health`
Cloud Storage	`reconcile-poc-29056-reconcile-raw-archive`, 7-day retention, not locked
Cloud SQL	`reconcile-state`, Postgres 16, private IP only
Pub/Sub	`run-complete`, `engine-requests`, `notifications`, each with a dead-letter topic
Service accounts	ingest, dbt, valuerecon, publish, engine, executor, agents, api, build
Other	VPC + private service access, Artifact Registry, Secret Manager (DB password), Cloud Run `reconcile-console`

5. Done. Terraform applied (119 resources planned); console deployed and `/api/health` returns ok for BigQuery and Cloud SQL; synthetic data for 11–13 Oct; CI green on `0a08808` (5 of 5 checks).

6. Exit gate.

- **To do** — Terraform builds the environment from scratch (destroy + apply) — apply done, rebuild proof pending
- **Done** — Generator loads a business date
- **Done** — Console on Cloud Run, `/api/health` ok
- **Done** — CI green

7. Issues and fixes. Base APIs had to be enabled before the first apply. Another project's `GOOGLE_APPLICATION_CREDENTIALS` was unset in the venv. `deploy_console.sh` temp path fixed for Windows (`7ea0254`). Company network blocks certificate-revocation checks: push from PowerShell, `curl --ssl-no-revoke`.

8. Risks and differences. Single project instead of three; no VPC Service Controls or IAP (trial account). Archive retention blocks `tf-destroy` for 7 days after any load.

Phase 1 — Ingest, control totals and raw archive

1. Status and exit gate. Implemented and passed 21 Oct 2026 (G1–G5 for 11, 12 and 13 Oct). CI green on `3e37dac`.

2. Goal and why. Every extract lands idempotently with a batch ID, writes its own receipt from the same snapshot, and is archived write-once. Phase 2 needs a trustworthy receipt; Phase 4 needs an untouched copy to replay; retries must never double data. *Example:* 13 Oct encounters land 10,012 rows as batch `b-20261013-encounters-04`; `ops.control_totals` says 10,012 rows and 10,012 IDs; the Parquet file sits under `raw/encounters/load_date=2026-10-13/`.

3. Design. Five steps per table and day:

1. **Probe** the source and write `ops.source_health`.
2. **Extract** rows and totals (rows, distinct keys, null amounts, ₹ total) in one read-only, repeatable-read transaction.
3. **Archive** Parquet + manifest with SHA-256 to the write-once bucket.
4. **Load** in one BigQuery transaction: delete the day, insert the batch, write the receipt, log the batch.
5. **Announce** `ingest.run_complete` on Pub/Sub for the Phase 4 engine.

The business date is the India (IST) facility day; source timestamps are UTC. Batch IDs include the table name so tables load in parallel without a shared counter.

4. GCP infrastructure. No new resources: uses the Phase 0 archive bucket, `raw` and `ops` datasets, `run-complete` topic and `sa-ingest`. Airflow DAGs `ingest_ehr`, `ingest_lis`, `ingest_billing` (01:00 IST) and `replay_batch`; an Airflow pool `bq_load` serialises loads.

5. Done. Three dates loaded; latest receipts per day: 11 Oct 61,252 rows, 12 Oct 76,227, 13 Oct 77,639. 11 Oct is short because its 00:00–05:30 IST slice falls on 10 Oct UTC, which was not generated; use 12 and 13 Oct as normal days. Injectors built: `drop_hour`, `duplicate_rows`, `missing_control_totals`.

6. Test cases and exit gate. T1 (same date twice), T2 part 1 (missing receipt injector), T9 part (ingest denied on `analytics`).

- **Done** — G1 exactly one receipt per batch (9 of 9 datasets)
- **Done** — G2 re-running a date gives identical counts
- **Done** — G3 an archived file replays into raw
- **Done** — G4 ingest cannot write `analytics`
- **Done** — G5 archive objects cannot be overwritten (412) or deleted (403)

7. Issues and fixes. BigQuery load failed with *decimal target types and schema*; removed `decimal_target_types` (`3e37dac`). Airflow

had no Google login on Windows; compose now mounts `${GCLOUD_CONFIG_DIR}`. `make verify` must run only after all three DAG runs are green. Two unit tests fail only on Windows.

8. Risks and differences. `make trigger` unpauses DAGs, and a scheduled 01:00 IST run then loads a date with no data; pause the three DAGs after manual runs. `ops.control_totals` keeps one receipt per batch, so count a day using the latest receipt per date and dataset.

Phase 2 — dbt models and reconciliation checks

1. Status and exit gate. Implemented and passed 22 Oct 2026 per runbook v1.2. The console Reconciliation screen, results collector and `/api/reconciliation` were deferred to Phase 3.

2. Goal and why. Phase 1 records the receipt; Phase 2 checks the warehouse against it. Without these tests a green run proves nothing. *Example:* with `drop_hour` injected, Airflow was green but `reconcile_to_control_totals` failed: receipt 10,012 vs 9,895 landed, 117 encounters missing, naming the exact day and batch. A clean reload and rebuild passed again.

3. Design.

- **Staging** `stg_encounters`, `stg_claims`, `stg_lab_results`: `insert_overwrite` on the business date plus 2 days before (3-day look-back), `row_number()` dedupe, `full_refresh: false`.
- **Marts** `dim_encounters`, `fct_daily_claims` (joins encounters on purpose, so Phase 3 holds it when encounters is held), `fct_bed_occupancy`.
- **Receipt checks**: `reconcile_to_control_totals` (rows, distinct IDs, empty amounts, ₹ total against the latest receipt) and `control_totals_present` (exactly one receipt per batch). Both are needed: without the second, a missing receipt silently compares against an older one.
- **Quality**: raw duplicate warning, mart-vs-staging test, lab unit and range check (`lab_reference_ranges` model).
- **Safety**: `policy_outcome: blocking` by default; `assert_expected_tests` on-run-end fails the run if any test on a built model was skipped.
- **Value level**: each `result_value`, `result_unit`, `collected_at` is canonicalised and deterministically encrypted with a KMS-wrapped keyset; differences are written as record key and field name only.
- **Airflow**: `dbt_build` DAG with `dbt_debug` → `dbt_build_and_test` (as `sa-dbt`) → `value_recon` (as `sa-valuerecon`); dbt in its own venv in the image.

4. GCP infrastructure. New: Cloud KMS key ring `reconcile`, key `recon-digest` (asia-south1); role `roles/cloudkms.cryptoKeyEncrypterDecrypterViaDelegation` for `sa-valuerecon` only; wrapped keyset v1 in `evidence_restricted.recon_keyset`; `.env` gains `SA_VALUERECON`, `KMS_RECON_KEY`. Existing: `audit`, `evidence_restricted`, `sa-dbt`.

5. Done. Full build `PASS=49`; value-recon build `PASS=4`; dbt green from Airflow.

6. Test cases and exit gate.

Test	Injected	Caught by	Result
T3 missing hour	`drop_hour`	rows and IDs vs receipt	Proven 21 Oct
T2 missing receipt	`missing_control_totals`	`control_totals_present`	Proven 21 Oct
T4 raw duplicates	`duplicate_rows`	raw warning; staging dedupe	Proven 22 Oct
T5 offsetting 50 + 50	`offsetting_errors`	IDs and ₹ vs receipt	Proven 22 Oct
T11 unit change	`unit_change`	unit and range check	Proven 22 Oct
T6 14-row truncation	`inject_value_truncation`	KMS value check	Proven 22 Oct
T7 skipped test	`--exclude`	expected-test-count	Proven 22 Oct
No values in evidence	—	`evidence_has_no_values`	Proven 22 Oct

7. Issues and fixes. KMS `useToEncryptViaDelegation` denied with the plain encrypter role; fixed with the ViaDelegation role. On the valuerecon connection models defaulted to `evidence_restricted`; fixed with `+schema: audit`. `dbt debug` rejects `--target-path`; use `DBT_TARGET_PATH`. A pasted Dockerfile overwrote a DAG file; restored from git.

8. Risks, gaps and the recorded deviation.

- **Deviation from v4.** v4 writes value-level evidence through dbt `store_failures` on a test. The build writes it through the model `value_recon_lab_results`, replacing its 3-day window each run. Impact: permissions are the same (only `sa-valuerecon` writes `evidence_restricted`); retention is now the model's replace cycle, so a repaired day leaves no stale evidence but history is not kept; privacy is guarded by the `evidence_has_no_values` test on columns; v4's evidence-location test must be rewritten against the model's table. **Alignment rule:** the model, its `sa-valuerecon` permissions and every evidence test (location, columns, no values) are written against `value_recon_lab_results`; no v4 `store_failures` acceptance test is counted as evidence for the current build.
- **Gap.** Billing ingest does not re-extract recent days by itself; the look-back was proven with manual re-loads. Automate in ingest or via Phase 4 `lookback_rebuild`.
- KMS key rings are never deleted by Google; `tf-destroy` only disables the key.

Phase 3 — Publication gate and Write-Audit-Publish

1. Status and exit gate. Not started. Next phase to build.

2. Goal and why. Consumers only ever see verified days. Checks are useless if bad data still reaches reports; the gate turns "a test failed" into "this day is held, reports keep yesterday's verified figures, marked delayed". *Example:* encounters for 13 Oct is held for its count gap; `fct_daily_claims` passes its own checks but joins encounters, so it also stays on 12 Oct; claims and lab results publish normally.

3. Design.

- **Publication state** per dataset and business date in Cloud SQL: `PENDING` → `PUBLISHED`, `HELD`, or `PUBLISHED_WITH_CAVEAT` (only where dbt meta allows a caveat, within its maximum gap, approved by the named owner; never for a never-auto failure).
- **Results collector**: parses `run_results.json`, stored failures and recon results into Cloud SQL (carried over from Phase 2).
- **Gate rule**: publish only if every blocking test passed, reconciliation passed, every upstream dataset is published, and no open incident or source breaker holds the date. Reads `meta.policy_outcome`, not dbt severity.
- **Dependency map** from dbt `manifest.json`, so a mart waits for every upstream dataset.
- **Publish job**: re-reads state and open breakers immediately before each clone, takes a BigQuery table snapshot, then `CREATE OR REPLACE TABLE analytics.x CLONE audit.x` in dependency order. A mart counts as published only when all its tables are cloned.
- **Rollback**: restore the pre-publish snapshot, then re-run reconciliation on it.
- **Console and consumers**: Reconciliation screen (deferred from P2), Publication ledger (datasets × days), gate checklist, and a "Data delayed: last verified 12 Oct" banner; Looker Studio reads `analytics` only.

4. GCP infrastructure.

New	Purpose
Cloud Run job `publish` as `sa-publish`	Gate check + clone
Cloud SQL tables: publication state, test results, publish log	Gate decisions and history
BigQuery table snapshots	Rollback point per publish
Airflow task or DAG to trigger publish after `dbt_build`	Until the Phase 4 engine triggers it
Looker Studio report on `analytics`	Consumer view of verified data

Already in place: `analytics` writable only by `sa-publish` (since P0).

5. Build checklist.

- **To do** — Results collector and `/api/reconciliation`, Reconciliation screen
- **To do** — Publication state table and dependency map
- **To do** — Publish job with snapshot before clone
- **To do** — `/api/publication` endpoints and caveat request
- **To do** — Publication ledger and delayed banner

6. Test cases and exit gate. T2 part 2 (missing receipt holds publication), T8, T9.

- **To do** — A held dataset and all its dependants are never cloned
- **To do** — A write to `analytics` by any non-publish identity is denied and logged
- **To do** — A rollback restores the previous snapshot and re-runs reconciliation on it

7. Issues and fixes. None yet.

8. Risks and differences. BigQuery has no atomic schema swap, so publishing is table by table; the dependency rule stops a report mixing verified and unverified data. Caveat policy needs data-owner input.

Phase 4 — Decision engine and recovery executor

1. Status and exit gate. Not started. Depends on Phase 3 state and the results collector.

2. Goal and why. Turn failures into incidents, decide deterministically, run one safe action, and close only after re-verification. It replaces the manual L1 loop with a decision that is the same for the same input, so it can be audited and replayed. *Example:* the engine reads the failed encounter reconciliation. Gate 1: 42 of 42 tests ran. Gate 2: no outage group open. Gate 3: gap 8%, above the 0.1% tolerance, so not expected. Gate 4: all 800 rows in 02:00–03:00, so `partial_extract`. Gate 5: attempt 1 of 2. The executor replays the hour, rebuilds, re-verifies 10,000 vs 10,000 and publishes in about 12 minutes.

3. Design.

- **Incident key**: dataset, business date, failure type, scope. Signals with the same key join one incident.
- **Evidence contract**: the engine refuses to decide on incomplete evidence (snapshot, expected vs actual, failed checks, completeness, evidence reference, deployment context).
- **Five gates** and **three policy stages** as in Part A; severity P1–P3; mode auto-fix, approve or engineer fix; attempt cap 2.
- **Lifecycle**: DETECTED → TRIAGED → (AWAITING_APPROVAL) → REMEDIATING → VERIFYING → RESOLVED, or ESCALATED. Publication state is tracked separately.
- **Run-ID claim**: one atomic insert per run-complete event; lease, status (CLAIMED, COMPLETED, FAILED) and a takeover limit, so a redelivered or concurrent event is evaluated once.
- **Partition lock** with a lease, and an **action journal** written before and after each attempt.
- **Catalog actions** as dbt invocations: `reextract_from_raw`, `rebuild_partition`, `lookback_rebuild`, `link_unknown_member`,

`quarantine_rows`; orchestrator actions `wait_and_recheck`, `log_expected`, `hold_publish`.

- **Deployment context** from GitHub deployments attached as a clue, never as proof; `suspected_regression` when no data rule matches.
- **Stable interface for Phase 5**: incident schema, catalog list, and a policy re-check call that agents' proposals and approvals must pass through.

4. GCP infrastructure.

New	Purpose
Cloud Run service `engine` as `sa-engine`	Five gates, incidents
Cloud Run job `executor` as `sa-executor`	One catalog action under lock
Pub/Sub push subscriptions on `run-complete`, `engine-requests`	Feed the engine; dead letters keep lost events
Cloud SQL tables: incidents, run claims, locks, action journal	State and audit
Scheduled sweep (Cloud Scheduler or Airflow)	Re-publish expired claims
GitHub deployments read access	Deployment context

5. Build checklist.

- **To do** — Run-ID claims, partition locks and action journal
- **To do** — Engine: incident key, evidence contract, five gates, severity, mode, attempt cap
- **To do** — Executor with catalog actions; never-auto list enforced
- **To do** — `/api/incidents`, retry and kill-switch endpoints
- **To do** — Incidents list and detail screens with the gate trace

6. Test cases and exit gate. T10–T14.

- **To do** — Every row of the v4 failure-to-decision table gives the stated decision and mode
- **To do** — The same input replayed gives the same decision
- **To do** — A duplicate or concurrent event produces one evaluation and no second action

7. Issues and fixes. None yet.

8. Risks and differences. Phase 5 integration waits until the incident schema, catalog and re-check interface are agreed and tested here. The billing look-back gap from Phase 2 can be closed by `lookback_rebuild`.

Phase 5 — AI agents, approvals and notifications (frozen design)

1. Status and exit gate. Not started. Design frozen in v5. Agents stay disabled for real healthcare data until written residency, BAA and no-training assurances are recorded.

2. Goal and why. Advisory judgement for unknown failures and human approval for L2 fixes, without giving any agent the power to act. Rules cover known failures; agents shorten diagnosis of new ones; approvals keep data owners in charge of fixes that change meaning. *Example:* `lab_orders` arrives with no receipt and no rule explains why. Diagnosis cites the missing receipt row and a 401 from the lab system's token endpoint and proposes `retrigger_ingestion` (added to the catalog in this phase) at 0.86. The engine validates it and routes it through the gates; while agent-proposed actions require approval, Communication sends a Slack card, the owner approves, and the engine re-checks before the executor runs it.

3. Design.

Scope

Agent	In POC	Trigger	Output	Hard boundary
Diagnosis	Fully built, validated, evaluated	Unknown failure or low-confidence rule match	Confirmed facts with evidence references, hypotheses, one catalog action, confidence	No data writes, no commands, no PHI
Communication	Fully built, with Slack approvals	Alert, approval or escalation needed	Slack card, alert, ticket text with counts and references only	Cannot approve its own request; no PHI
Runbook	Match-only; PR drafting optional	Incident created or resolved	Link to the matching runbook	Cannot merge or enable rules
Code-fix	Excluded; separately gated later	Engineer-fix incidents	Tested PR in an isolated sandbox	No production write, merge or deploy

The engine is the sole supervisor. It decides when an agent is called, validates the output, and is the only path to the executor. Agents never call each other or the executor.

Agent input contract (minimal). Allowed: incident metadata and approved non-identifying context; counts, mismatch categories and opaque evidence references; the matching runbook. Not allowed: patient identifiers, raw clinical values, stored digests, or access to `evidence_restricted`. Any extra context must pass an allowlist and privacy check. An evidence reference is not permission to retrieve the evidence: agents cannot resolve restricted references themselves.

Output contract and validation. Strict JSON: `confirmed_facts[]` (each with an evidence reference), `hypotheses[]`, `proposed_action`, `confidence`. The engine checks schema, that each reference exists, that the action is in the catalog and not never-auto, and confidence ≥ 0.80. Log lines, runbooks and retrieved text sit in a delimited data section and are never instructions. Invalid output, timeout or platform failure → classified unknown and escalated; the pipeline never waits on an agent.

Approval state machine.

1. The engine creates an approval record in state `PENDING` with an expiry and a **proposal fingerprint**: a hash over incident ID, proposed action and parameters, evidence and version references, and policy version.
2. Communication posts the Slack card (counts and evidence reference only).
3. The Slack callback reaches a public endpoint that verifies the signing secret and timestamp, the allowed user and channel, and rejects replays.
4. The transition `PENDING → APPROVED | REJECTED | EXPIRED` is one atomic update; it succeeds only from `PENDING`.
5. On `APPROVED`, the engine recomputes the fingerprint and re-runs the policy. If the proposal, evidence or policy changed, the approval is rejected and a new decision requested. If the incident is already resolved or expired, the callback is rejected.
6. A duplicate callback returns the existing outcome; the action is never repeated.
7. `REJECTED` or `EXPIRED` → incident `ESCALATED`, publication stays `HELD`.

What is stored. Kept separate from `evidence_restricted`:

- *Agent provenance* per call: agent, model, prompt and policy versions; input evidence references; structured output; validation result; confidence; tokens, cost and time.
- *Approval record*: incident ID, proposed action, fingerprint, approver identity, decision, decision time, expiry, evidence reference, policy version.
- Rejections are stored too; they feed agent-quality tracking (accepted rate, human-confirmed rate, invalid outputs, timeouts, latency, cost) and a reviewed rollback of a degraded version.

Residency is enforced as a configuration gate, not only an instruction. A GCP region setting does not prove that model inference and processing happen in India, so enabling agents for real data needs a recorded assurance, checked in three places:

1. **Policy:** `agents.enabled` (and any real-data flag) may be `true` only when `policy.yml` records the written residency, BAA and no-training confirmations with a document reference. The CI policy check, which already rejects `agents.enabled: true`, fails the build otherwise.
2. **Infrastructure:** Terraform grants `sa-agents` model-invocation rights for a real-data environment only when the same confirmation reference is set; without it the binding is not created.
3. **Runtime:** the engine checks the flag before every agent call and treats a disabled platform as unavailable (escalate as unknown).

Until all three are in place with a real reference, agents run on synthetic data in the test project only.

Unknown becomes known only through review. A resolved incident can lead to a proposed rule as a pull request; it is replayed against past incidents and the injection suite, approved by a person, bumps the policy version, and starts recommend-only before it may auto-fix.

Evaluation is a release gate. A golden set of 30–50 synthetic incidents is scored for valid output, evidence references, correct action, consistency across repeated runs, injection resistance and unsafe-action rejection. A changed agent, model, prompt or context builder cannot merge if it scores worse than the approved baseline or gets an out-of-catalog action accepted. (The plan placed the CI gate in Phase 6; v5 moves it here.)

4. GCP infrastructure.

Item	Status	Detail
`sa-agents`	Exists (P0)	Grant only: model invocation, read on approved masked views, logs, runbooks. No `raw`, `audit`, `analytics`, `evidence_restricted` or KMS
Agent Platform API, Model Armor API	New	asia-south1; pinned Gemini model; Model Armor template on every prompt
Masked views	New	Views over `audit` with identifiers and clinical values masked
Public Slack callback service	New	Small Cloud Run service, unauthenticated ingress, signature and replay checks, rate limit; all other services stay private
Secret Manager	New entries	Slack signing secret, bot token
Slack app	New	Bot token, interactive buttons, allowed channel
Pub/Sub `notifications` + DLQ	Exists (P0)	New subscription for Communication
Cloud SQL tables	New	Agent provenance, approvals
Budgets	New	Token and time budget per call; AI spend alert
GitHub CI	New job	Golden-set evaluation and contract tests

5. Build checklist.

- **To do** — Written residency, BAA and no-training confirmation recorded (precondition for real data)

- **To do** — Phase 4 incident schema, catalog and re-check interface agreed and tested
- **To do** — Masked views and `sa-agents` permissions
- **To do** — Diagnosis and Communication agents behind Model Armor; Runbook match-only
- **To do** — Output validation and failure path
- **To do** — Approval state machine, fingerprint, public callback service
- **To do** — Provenance and approval tables; `/api/agents/proposals`, `/api/approvals`
- **To do** — Golden set and evaluation gate in CI
- **To do** — Agents and approvals screen

6. Test cases and exit gate. A test group counts as passed only when every test ID in it has a recorded output at its evidence location. Existing code or fixtures are not evidence. New test IDs T22–T27 extend the v4 list T1–T21.

Test group	Test IDs	Required result	Evidence location (planned)	Status
Output contract	T15	Malformed JSON, missing evidence and out-of-catalog actions rejected; incident escalated	CI contract-test report; engine validation log entries	Not run
Prompt injection	T16	A log line instructing `full_refresh` (or any instruction in data) cannot override policy	CI contract-test report; rejected proposal in the provenance table	Not run
Agent unavailable	T17	Timeout or platform error escalates as unknown; run not delayed	Incident record with escalation reason and timestamps	Not run
Provenance	T22	Every call records agent, model, prompt and policy versions with input references	Agent provenance table in Cloud SQL (one row per call)	Not run
Slack approval	T18, T23	Approve, reject, timeout, expired, invalid signer and duplicate callback each behave as designed; remap only after approval	Approval table rows and callback-service logs	Not run
Approval integrity	T24	An approval cannot authorise a changed or stale proposal (fingerprint mismatch, resolved incident)	Approval table; engine re-check log	Not run
Golden set	T25	Scores at or above the approved baseline; no out-of-catalog action accepted	CI evaluation report per agent version	Not run
Privacy	T26	No PHI or clinical values in prompts, messages or tickets; `sa-agents` denied on unmasked tables and `evidence_restricted`	Scan report; IAM denial entries in Cloud Audit Logs	Not run
Kill switch	T27	With the kill switch on, agent proposals become recommendations only; no action starts	Incident record and action journal (no executor entry)	Not run
Residency and agreements	Precondition	Written confirmations recorded and referenced in `policy.yml` before real data reaches the model platform	Signed documents; policy reference; CI policy check	Not received

7. Issues and fixes. None yet.

8. Risks and differences. Without residency confirmation, agents run on synthetic data only and unknown cases go to a human. The design's separate agents project and VPC Service Controls are not available on the trial account. The Slack callback is the only internet-reachable endpoint.

Phase 6 — Outage breaker, hardening and demo

1. Status and exit gate. Not started.

2. Goal and why. Handle a correlated source outage as one incident, run the full injection suite, and present the four demos with measured results. Twenty per-dataset fixes against a broken source would hammer it and flood on-call. *Example:* the lab interface is taken down; `lab_results`, `lab_panels` and `lab_orders` fail within 30 minutes; the breaker opens, holds all three and two dependent marts, and sends one alert. Billing and EHR keep publishing. When the probe and freshness both pass, each lab dataset re-runs the five gates, oldest date first.

3. Design.

- **Breaker (LIS in the POC)**: opens at 3 failing datasets or 50% of the source's inventory within 30 minutes; holds the source's datasets and dependants; cancels queued per-dataset fixes; one parent incident; closes only when the source probe and freshness both pass. A fix already running finishes and is journaled but cannot publish (publish-time recheck). One source's breaker never stops another source.
- **Global breaker** at 2 failing sources switches all automation to recommend-only; healthy sources still publish.
- **Injection suite** for every acceptance case: missing hour, duplicates, offsetting errors, missing totals, unit change, value truncation, archived parent, merged patient, schema change, interface down, replay in flight during a breaker.
- **KPIs**: auto-recovery rate, automated MTTR, stale-data duration, publication blocks, false alarms, by response mode.

4. GCP infrastructure.

New	Purpose
Cloud Monitoring uptime checks on engine and executor	Fallback alerting independent of Slack
Budget alerts and per-service-account BigQuery quotas	Cost guardrails
Audit-log sink to a locked bucket	Tamper-evident audit trail
`/api/metrics`, `/api/outages`, `/api/injections` (test project only)	Operations screen and demos

5. Build checklist.

- **To do** — Breaker in the engine
- **To do** — Full injector suite with assertions
- **To do** — Fallback alerting, budgets, quotas, audit sink
- **To do** — Metrics and outages APIs; Operations screen

6. Test cases and exit gate. T19–T21.

- **To do** — All four demo scenarios pass live in the console
- **To do** — The injection suite shows zero bad publishes
- **To do** — KPI baseline captured against today's manual process

7. Issues and fixes. None yet.

8. Risks and differences. Demo scenario 2 in the POV deck ("lab feed returns 0 rows → agent → human approves") would normally match the stale-feed rule; align it with the Phase 5 example (missing receipt plus a 401) so the case is genuinely unknown.

Part C — Tracking

Verification tracker (v5)

An item is marked proven only from a recorded test run, never from a successful build. Overall labels sit on the group; individual tests keep their own status.

Tracker item	Individual test	v5 status	Evidence
Value-level SQL and macros — **Partly verified**	14-row truncation detected	Proven	Runbook v1.2, Phase 2 demo T6
	NULL vs literal text distinction	Not yet run	—
	Case-sensitive and case-insensitive fields	Not yet run	—
	Malformed transform fails at compile time	Not yet run	—
	Duplicate target key reported	Not yet run	—
Evidence contents and privacy — **Partly verified**	Evidence holds keys and field names only	Proven	`evidence_has_no_values` pass
	Evidence location test (rewritten for the model's table)	Not yet run	—
	Slack, tickets and prompts carry no values	Not yet run (needs P5)	—
KMS permissions and key access — **Partly verified**	`sa-valuerecon` can use the key	Proven	Value-recon build `PASS=4`
	Other identities denied the key	Not yet run	—
KMS key and keyset rotation	Both rotation tests	Not yet run	—
Outage breaker	Six breaker tests	Not verified (P6)	—
Deployment context	Two tests	Not verified (P4)	—
Agent provenance, contract, injection, unavailable	T15–T17, T22 (see Phase 5 matrix)	Not verified (P5)	—
Evaluation gate and golden set	Golden set built, gate test	Not verified (P5)	—
Threshold approval	Approval test	Not verified	—
Duplicate events, policy validation	Five claim tests; invalid-policy test	Not verified (P4); policy CI check exists	CI policy check passes on valid policy
AI residency and agreement coverage	Written confirmation	Not received	—

SLOs (proposed, not agreed)

SLO	Target	Clock	Measured from
Detection	Within 5 min	Run completion → incident opened	P4
Automated recovery (L1)	Fixed and re-verified within 30 min, else escalated	Detection → re-reconciliation passes	P4
P1 notification	Within 5 min	Detection of a P1	P5
Data freshness	Business deadline met or flagged delayed	Agreed per dataset	P3
Publication safety	Zero publishes of failed data	Hard rule	P3
Approval wait, L2/L3 resolution	Measured only	Request → decision or fix	P5

Open decisions

Decision	Owner	Status
AI residency, BAA and no-training confirmation	Security and privacy	Open
Deterministic encryption vs HMAC	Security and privacy	Open
Whether agent-proposed actions always need approval during the POC	Data owner, platform owner	Proposed: yes
L2/L3 response targets	SRE and support	Open
Freshness deadline per dataset	Data owners	Open
Real volumes for thresholds	Data engineering	Open

Change log

Date	Version	Change
22 Oct 2026	v5.1	Precedence note at the start of Part B; Phase 5 test-to-evidence matrix with new test IDs T22–T27; residency enforced as a policy, Terraform and runtime gate; Phase 2 deviation alignment rule
22 Oct 2026	v5	Added per-phase plans with build status; components mapped to phases and the POV deck; Phase 5 frozen (scope, input contract, approval state machine with fingerprint, storage, evaluation gate, ten-group exit gate); Phase 2 deviation recorded; verification tracker updated from recorded evidence; four-state status model
15 Oct 2026	v4	Outage breaker, deployment context, value-level reconciliation, agent provenance, learned thresholds, pre-pilot essentials; policy 1.7.0